LP3 Viva Questions and Answers

SPPU Fourth Year Computer Engineering

Laboratory Practice III (LP3)

Group A	Group B	Group C
Design and Analysis of Algorithms	Machine Learning	Blockchain Technology

Simple, exam-oriented answers for practical viva voce.
Questions marked with ★ are commonly asked in practical vivas; this is a study aid and does not predict the actual examiner's questions.

Table of Contents

Click any entry to jump to that section.

1. How to Prepare for the LP3 Viva 4

2. Group A: Design and Analysis of Algorithms 5

A1. Recursive and Non-Recursive Fibonacci Numbers 5
A2. Huffman Encoding (Greedy Strategy) 6
A3. Fractional Knapsack (Greedy Method) 7
A4. 0/1 Knapsack (Dynamic Programming / Branch and Bound) 8
A5. N-Queens Problem (Backtracking) 9
A6. Deterministic and Randomized Quick Sort 10
A7(a). Mini Project: Matrix Multiplication and Multithreaded Matrix Multiplication 11
A7(b). Mini Project: Merge Sort and Multithreaded Merge Sort 12
A7(c). Mini Project: Naive String Matching and Rabin-Karp Algorithm 13
A7(d). Mini Project: Exact and Approximation Algorithms for the Travelling Salesperson Problem (TSP) 14

3. Group B: Machine Learning 16

B0. Machine Learning Basics (Common to All ML Practicals) 16
B1. Uber Fare Prediction (Linear Regression and Random Forest Regression) 17
B2. Email Spam Classification (KNN and SVM) 18
B3. Bank Customer Churn Prediction (Neural Network) 19
B4. Gradient Descent to Find the Local Minimum of a Function 20
B5. KNN Classification on the Diabetes Dataset 20
B6. K-Means and Hierarchical Clustering (Elbow Method) 21
B7. Stock Market Analysis and Prediction 22
B8. Titanic Survival Prediction 23
B9. Signature Identification Using a Custom Dataset 24

4. Group C: Blockchain Technology 26

C0. Blockchain Fundamentals (Common to All Blockchain Practicals) 26
C1. Installing MetaMask and Studying Ether Spent per Transaction 26
C2. Creating a Cryptocurrency Wallet Using MetaMask 27
C3. Bank Account Smart Contract (Deposit, Withdraw, Balance) 27
C4. Solidity Student Data Program (Structure, Array, Fallback) and Gas Analysis 28
C5. Types of Blockchains and Real-World Use Cases 28
C6. Creating a Business Network Using Hyperledger 29
C7. Blockchain-Based E-Voting dApp 29
C8. Blockchain-Based Transparent Charity Application 29
C9. Blockchain-Based Medical Records Application 30
C10. Blockchain-Based Mental Health Application 30

5. Algorithm and Model Comparison Tables 31

Group A: DAA comparisons . . . 31
Group B: Machine Learning comparisons . . . 32
Group C: Blockchain comparisons . . . 34

6. Important Formulas and Complexity Cheat Sheet . . . 36
Group A: Algorithms . . . 36
Group B: Machine Learning . . . 37
Group C: Blockchain . . . 37

7. Practical Demonstration Questions . . . 38
What to Do If the Examiner Asks You to Explain Your Program . . . 38
Common Follow-up Questions . . . 42
Questions for Mini Projects . . . 42

8. Top 50 Most Important Viva Questions . . . 44

9. Rapid-Fire One-Line Questions and Answers . . . 46
Group A: DAA . . . 47
Group B: Machine Learning . . . 49
Group C: Blockchain . . . 50

10. Last-Minute Revision Sheet . . . 51
Algorithm complexity at a glance . . . 51
Machine learning at a glance . . . 51
Blockchain at a glance . . . 51
Viva-day checklist . . . 51

1. How to Prepare for the LP3 Viva

- **Know your own code.** The examiner usually starts from your program, so be able to explain every function and the output.
- **Learn the 3-part answer:** definition, how it works, complexity or metric. Practise saying each in 10-20 seconds.
- **Memorise complexities** for every algorithm (see the cheat sheet). Be ready to explain why.
- **For ML, know your dataset:** number of rows, features, target, what preprocessing you did and which metric you used.
- **For blockchain, know your deployment:** which network, which account, what the gas used and fee were, and the transaction hash.
- **Run your program again** just before the viva so you can demonstrate it. Keep sample inputs ready, including one edge case.
- **Be honest.** If you do not know, say what you do know about the topic. Never invent results or accuracy values.
- **Use the ★ marks** in this guide to prioritise. A ★ means the question is commonly asked in practical vivas; it is not a guarantee of the actual exam.
- **Revise in this order:** Top 50 → Rapid-fire → Formulas and complexity sheet → Last-minute revision sheet.

What this guide contains

Part	Content	Count
Group A	Design and Analysis of Algorithms: 6 assignments + 4 mini projects	168 questions
Group B	Machine Learning: basics + 9 assignments	136 questions
Group C	Blockchain Technology: basics + 10 assignments	67 questions
Comparison tables	10 each for DAA, ML and Blockchain	30 tables
Cheat sheets	Formulas, complexity, demonstration templates, Top 50, rapid-fire, revision	extra

*Question IDs: **A-** Group A, **B-** Group B, **C-** Group C, **D-** demonstration follow-ups, **M-** mini-project questions, **T-** Top 50, **R-** rapid-fire. **Language:** algorithm explanations are language-neutral; where a library or API is named, the language is stated. **Datasets:** dataset names and columns are the usual ones for these practicals and may differ in your copy; this guide never quotes accuracy values, so use your own results. **Blockchain:** test networks use free test Ether, mainnet uses real Ether, and gas fees vary with network and congestion.*

2. Group A: Design and Analysis of Algorithms

A1. Recursive and Non-Recursive Fibonacci Numbers

★ A-1 What is the aim of this practical?

To write a program that generates Fibonacci numbers using **recursion** and using a **loop (non-recursive)**, and to compare their **time and space complexity**.

A-2 What is the Fibonacci sequence?

A sequence where each number is the sum of the previous two: 0, 1, 1, 2, 3, 5, 8, 13, 21 ...
F(0) = 0, F(1) = 1, F(n) = F(n-1) + F(n-2).

A-3 What problem does the algorithm solve?

It finds the n-th Fibonacci number (or prints the first n terms). It is mainly used to demonstrate how the algorithm style (recursion vs iteration) changes performance.

A-4 Which algorithmic strategy is used?

Recursive version: plain **recursion** (brute-force style, no reuse of results). Non-recursive version: **bottom-up iteration**, which is the idea behind **dynamic programming**.

A-5 Explain the recursive algorithm in steps.

1. If n is 0 return 0; if n is 1 return 1 (base cases).
2. Otherwise return fib(n-1) + fib(n-2).
3. Calls keep splitting until base cases are reached, then results are added back up.

A-6 Explain the non-recursive algorithm in steps.

1. Keep two variables a = 0 and b = 1.
2. Repeat n times: next = a + b; a = b; b = next.
3. At the end, a holds F(n).

★ A-7 What is the time complexity of the recursive Fibonacci?

O(2^n) (more exactly Θ(φ^n), where φ ≈ 1.618). It is exponential, so it becomes very slow for n around 40 or more.

★ A-8 Why is the recursive version so slow?

It **recomputes the same subproblems** again and again. For example, fib(5) calls fib(3) twice and fib(2) three times. Recurrence: T(n) = T(n-1) + T(n-2) + O(1).

★ A-9 What is the space complexity of the recursive version?

O(n), because the call stack goes n levels deep before reaching the base case.

★ A-10 What are the time and space complexity of the non-recursive version?

Time **O(n)** (one loop of n steps). Space **O(1)** (only two or three variables).

A-11 Can Fibonacci be computed faster than O(n)?

Yes. **Matrix exponentiation** or **fast doubling** gives **O(log n)** time. **Memoization** (storing results) gives O(n) time with O(n) extra space.

A-12 What is memoization? How is it different from tabulation?

Memoization is **top-down**: recursion plus a table that stores results already computed. Tabulation is **bottom-up**: fill a table from F(0) upward using a loop. Both reduce Fibonacci to O(n).

A-13 Why did you choose to implement both versions?

To show that a simple, elegant recursive program can be very inefficient, while an iterative program is fast and memory-friendly for the same output.

A-14 What are the advantages and disadvantages of each version?

Recursive: short, matches the mathematical definition; but exponential time and stack overflow risk.
Non-recursive: fast, constant space; slightly less direct to read.

A-15 What are the edge cases?

n = 0 (output 0), n = 1 (output 1), negative n (invalid input, show an error), and very large n (integer overflow in C/C++/Java; Python integers do not overflow).

A-16 What happens if n is very large in the recursive program?

It becomes extremely slow and may crash with **stack overflow** (`RecursionError` in Python, default limit about 1000 calls).

A-17 What output should the program produce?

For n = 10 terms: **0 1 1 2 3 5 8 13 21 34**. The 10th Fibonacci number F(10) is 55. Both versions must give the same values.

A-18 How do you measure running time in your program?

Record time before and after the function call using a timer (for example `time.perf_counter()` in Python, `chrono` in C++, `System.nanoTime()` in Java) and print the difference. Use the same n for both versions.

A-19 How many function calls does recursive fib(n) make?

Exactly 2·F(n+1) − 1 calls. For n = 2 that is 3 calls: fib(2), fib(1), fib(0).

A2. Huffman Encoding (Greedy Strategy)

★ A-20 What is the aim of this practical?

To implement **Huffman encoding**, a **greedy** algorithm that builds variable-length binary codes to **compress data without losing information**.

A-21 What is Huffman coding?

A method that gives **short codes to frequent characters** and **long codes to rare characters**, so the total number of bits needed is minimised.

★ A-22 Which greedy choice does Huffman use?

At every step, **pick the two nodes with the smallest frequencies** and merge them into a new node whose frequency is their sum.

A-23 Explain the algorithm in simple steps.

1. Count the frequency of each character.
2. Create a leaf node for each character and put all in a **min-heap**.
3. Remove two smallest nodes, merge them under a new parent (sum of frequencies), push it back.
4. Repeat until one node (the root) remains.
5. Walk the tree: **left edge = 0, right edge = 1**; the path to each leaf is its code.

★ A-24 What is a prefix code? Why is it important?

A prefix code is one where **no code is the beginning of another code**. This makes decoding unambiguous, as you can read bits until you hit a leaf.

A-25 Which data structure is used and why?

A **min-heap (priority queue)**, because we repeatedly need the two smallest-frequency nodes, and a heap gives each in O(log n).

★ A-26 What is the time and space complexity?

Time **O(n log n)** for n distinct characters (n−1 merges, each with O(log n) heap operations). Space **O(n)** for the heap and tree.

A-27 What is the difference between fixed-length and variable-length coding?

Fixed-length (like ASCII, 8 bits per character) uses the same bits for every symbol. Variable-length (Huffman) uses fewer bits for common symbols, giving smaller output.

A-28 Is Huffman coding lossless or lossy?

Lossless. The original data can be recovered exactly by decoding.

★ A-29 Give a small example.

Frequencies A=5, B=9, C=12, D=13, E=16, F=45 give codes like F=0, C=100, D=101, A=1100, B=1101, E=111. Total = 45·1 + 12·3 + 13·3 + 5·4 + 9·4 + 16·3 = **224 bits**, against 300 bits for 3-bit fixed codes.

A-30 Is the Huffman code unique?

No. When frequencies tie, different tie-breaking gives different trees, but the **total encoded length is always the same (optimal)**.

A-31 How do you decode a Huffman-coded message?

Start at the root. Read each bit: 0 means go left, 1 means go right. On reaching a leaf, output that character and return to the root. Continue until all bits are consumed.

A-32 Why does the greedy approach give an optimal answer here?

Huffman has the **greedy-choice property** (the two rarest symbols can safely be siblings at the deepest level) and **optimal substructure** (merging them leaves a smaller problem of the same kind).

A-33 How many internal nodes does the Huffman tree have for n characters?

n − 1 internal nodes, because it is a full binary tree with n leaves.

A-34 What are the edge cases?

Empty input, only **one unique character** (give it code 0 so it has at least 1 bit), and all characters with equal frequency (codes become almost fixed length).

A-35 What are the advantages and disadvantages?

Advantages: optimal prefix code, simple, lossless.
Disadvantages: needs frequencies first (two passes), the tree/table must be stored or sent, and each symbol uses a whole number of bits.

A-36 Where is Huffman coding used in real life?

In compression formats such as **ZIP/GZIP (DEFLATE)**, **JPEG** and **MP3** (as one stage of compression).

A-37 How is Huffman different from Shannon-Fano coding?

Huffman builds the tree **bottom-up** from the least frequent symbols and is always optimal. Shannon-Fano splits **top-down** and is not always optimal.

A-38 How do you compute the average code length?

Σ (frequency of symbol × code length) / Σ frequencies. In the example above, 224 / 100 = 2.24 bits per symbol.

A3. Fractional Knapsack (Greedy Method)

★ A-39 What is the aim of this practical?

To solve the **Fractional Knapsack problem** with a **greedy** method and find the maximum total value that fits in a bag of given capacity.

A-40 What is the Fractional Knapsack problem?

Given n items, each with a value v and weight w, and a bag of capacity W, choose items (or parts of items) to **maximise total value** without exceeding W. Items can be **broken into fractions**.

★ A-41 What is the greedy criterion?

Sort items by **value-to-weight ratio (v/w) in descending order** and keep taking the best-ratio items.

A-42 Explain the algorithm in steps.

1. Compute v/w for every item.
2. Sort items by ratio, highest first.
3. For each item: if it fits completely, take it all and reduce remaining capacity; otherwise take the fraction (remaining capacity / w) and stop.

★ A-43 What is the time and space complexity?

Time **O(n log n)** because of sorting (the loop is O(n)). Extra space O(1) apart from storing ratios or the sorted list (O(n)).

★ A-44 Solve a small example.

W = 50; items (value, weight): (60,10), (100,20), (120,30). Ratios: 6, 5, 4. Take item 1 (10 kg), item 2 (20 kg), then 20/30 of item 3 = 80. **Total = 60 + 100 + 80 = 240**.

A-45 Why is the greedy method optimal here?

Because fractions are allowed, the bag can always be filled completely with the highest value-per-kg material, so no better choice exists (proved by an exchange argument).

A-46 Why not sort by value alone or weight alone?

A high-value item may be very heavy and waste capacity; a light item may have low value. Only the **ratio** measures value per unit of capacity.

A-47 Does the greedy method work for 0/1 Knapsack?

No. For the same example with 0/1 choice, greedy picks items 1 and 2 (value 160), but the optimal is items 2 and 3 (value **220**).

A-48 What are the edge cases?

Capacity 0 (answer 0), total weight of all items less than W (take everything), an item with weight 0 (take it, it costs nothing), negative values or weights (invalid).

A-49 What are the advantages and disadvantages?

Advantages: simple, fast, provably optimal for the fractional version.
Disadvantages: works only if items are divisible; not valid for 0/1 Knapsack.

A-50 Where is it applied in real life?

Loading **divisible goods** (grain, liquids, fuel), resource and budget allocation, and cutting stock problems where partial use is allowed.

A-51 What output does the program give?

The maximum value (240 in the example above) and usually how much of each item was taken, for example item 3 taken as 0.667.

A-52 Can the sorting step be avoided?

In theory the **weighted-median / selection** method gives O(n) time, but sorting is simpler and is what is normally implemented in labs.

A4. 0/1 Knapsack (Dynamic Programming / Branch and Bound)

Either dynamic programming or branch and bound may be used in the lab. Both are covered below.

★ A-53 What is the aim of this practical?

To solve the **0/1 Knapsack problem** using **dynamic programming (DP)** or **branch and bound (B&B)** and find the maximum value for a given capacity.

A-54 What is the 0/1 Knapsack problem?

Choose from n items, each with value v and weight w, to maximise total value within capacity W. Each item is **either taken completely (1) or left (0)**; no fractions.

★ A-55 Why can't a greedy method solve it?

Taking the best ratio first can leave unused space that no remaining item fits. Example: W=50, items (60,10), (100,20), (120,30): greedy gives 160, optimum is 220.

★ A-56 Explain the DP approach.

Build a table dp[i][w] = best value using the first i items with capacity w. Fill it row by row; the answer is dp[n][W]. It works because the problem has **optimal substructure** and **overlapping subproblems**.

★ A-57 Write the DP recurrence.

If w(i) > w: **dp[i][w] = dp[i-1][w]**.
Else: **dp[i][w] = max(dp[i-1][w], v(i) + dp[i-1][w - w(i)])**.
Base: dp[0][w] = 0 and dp[i][0] = 0.

★ A-58 What is the time and space complexity of the DP solution?

Time **O(n × W)** and space **O(n × W)**. Space can be cut to **O(W)** using a single 1-D array.

A-59 Why is O(nW) called pseudo-polynomial?

Because it depends on the **numeric value** of W, not on the number of bits needed to write W. If W is huge, the running time is huge.

A-60 How do you find which items were selected?

Backtrack from dp[n][W]: if dp[i][w] ≠ dp[i-1][w], item i was taken, so subtract its weight and move up; otherwise item i was not taken.

A-61 Small example?

W = 8; items (value, weight): (10,5), (40,4), (30,6), (50,3). Best choice: items 2 and 4, weight 7, **value 90**.

A-62 In the 1-D DP version, why does the weight loop run backwards?

So each item is used **only once**. Going forwards would let the same item be counted again for a smaller capacity (that becomes the unbounded knapsack).

★ A-63 Explain the Branch and Bound approach.

Build a state-space tree where each level decides take / not take an item. For each node, compute an **upper bound** (usually by solving the fractional knapsack for the remaining items). If the bound is not better than the best answer found so far, **prune** that branch.

A-64 What is the complexity of Branch and Bound for 0/1 Knapsack?

Worst case O(2^n), but pruning usually makes it much faster in practice.

A-65 What is brute force for this problem and its cost?

Try all 2^n subsets and check weight and value. Time **O(2^n)**, so it is only feasible for tiny n.

A-66 What are the edge cases?

An item heavier than W (can never be taken), W = 0, n = 0, items with zero value, and several optimal solutions with the same value.

A-67 Is 0/1 Knapsack NP-complete?

Its **decision version is NP-complete**. DP still solves it in pseudo-polynomial time.

A-68 What are the advantages and disadvantages of DP here?

Advantages: always optimal, easy to trace items.
Disadvantages: slow and memory-heavy when W is very large; weights must be integers (or scaled).

A-69 Real-life applications?

Project selection under a budget, cargo loading, choosing investments, and selecting features or files to fit a storage limit.

A-70 What output do you display?

The maximum value, the chosen items (and their weights), and optionally the DP table.

A5. N-Queens Problem (Backtracking)

★ A-71 What is the aim of this practical?

To place **N queens on an N × N chessboard** so that no two queens attack each other, using **backtracking**.

A-72 What are the rules of the N-Queens problem?

No two queens may share the **same row, same column, or same diagonal**.

★ A-73 What is backtracking?

A systematic trial-and-error method: build a solution step by step, and when a partial solution cannot lead to a valid answer, **undo the last step and try another option**.

★ A-74 Explain the N-Queens algorithm in steps.

1. Start with row 0.
2. For each column in this row, check if placing a queen is **safe**.
3. If safe, place it and recurse to the next row.
4. If no column works, **backtrack** to the previous row and move that queen.
5. When all N rows are filled, a solution is found.

★ A-75 How do you check whether a position is safe?

Column: no queen already in that column.
Diagonals: for queens at (r1,c1) and (r2,c2), they attack if **|r1 − r2| = |c1 − c2|**.
Rows are never clashing because we place one queen per row.

★ A-76 What is the time complexity?

Worst case about **O(N!)**, since row 1 has N choices, the next at most N−1 safe choices, and so on. Pruning makes real running time much lower.

A-77 What is the space complexity?

O(N) for the recursion stack and a column-position array ($O(N^2)$ if a full board matrix is stored).

A-78 How many solutions exist for small N?

N=1: 1, N=2: 0, N=3: 0, N=4: 2, N=5: 10, N=6: 4, N=7: 40, **N=8: 92** (12 distinct if rotations/reflections are treated as the same).

A-79 Show the solutions for 4-Queens.

Using column numbers per row (1-based): **(2, 4, 1, 3)** and **(3, 1, 4, 2)**.

A-80 How can the safe check be made O(1)?

Keep boolean arrays for columns and diagonals. On one diagonal direction **r − c is constant**, on the other **r + c is constant**. Bitmasks make it even faster.

A-81 What is a state-space tree?

A tree where each node is a partial solution (some queens placed) and each child adds one more queen. Backtracking explores it depth-first and cuts off invalid branches.

A-82 Why is backtracking better than brute force here?

Brute force would place all N queens and then check (N^N or $C(N^2, N)$ possibilities). Backtracking **abandons a branch as soon as a conflict appears**, saving huge work.

A-83 What are the edge cases?

N = 1 (one solution), N = 2 and N = 3 (no solution; the program should say so), very large N (finding all solutions becomes very slow), invalid $N \leq 0$.

A-84 What is the difference between finding one solution and all solutions?

For one solution, stop at the first complete placement. For all solutions, continue after printing each one by removing the last queen and trying the next column.

A-85 Applications of N-Queens / backtracking?

Constraint-satisfaction problems such as **Sudoku, graph colouring, scheduling**, and as a standard teaching example of backtracking.

A-86 What output does the program print?

A board where **Q** marks a queen and **.** an empty cell (or a list of column positions), and the total number of solutions if all are required.

A6. Deterministic and Randomized Quick Sort

★ A-87 What is the aim of this practical?

To implement **Quick Sort** with a fixed (deterministic) pivot and with a **randomly chosen pivot**, and compare their analysis and running time.

★ A-88 What is Quick Sort?

A **divide-and-conquer** sorting algorithm: choose a **pivot**, partition the array so smaller elements are on its left and larger on its right, then recursively sort both parts.

★ A-89 Explain Quick Sort in steps.

1. Choose a pivot.
2. **Partition**: rearrange so elements ≤ pivot come before it and elements > pivot come after.
3. The pivot is now at its final position.
4. Recursively sort the left and right sub-arrays.
5. Stop when sub-array size is 0 or 1.

A-90 What is deterministic Quick Sort?

The pivot is chosen by a fixed rule, such as always the **first, last or middle element**. The same input always follows the same path.

★ A-91 What is randomized Quick Sort?

The pivot is picked **uniformly at random** from the current sub-array (usually swap it with the last element, then partition as usual). No fixed input is always bad.

★ A-92 What are the time complexities?

Best: O(n log n). Average: O(n log n). Worst: O(n²). Randomized Quick Sort has **expected time O(n log n)** for every input.

★ A-93 When does the worst case occur?

When the pivot is always the smallest or largest element, for example an **already sorted or reverse sorted** array with first/last pivot. Partitions become sizes 0 and n−1.

A-94 Write the recurrence relations.

General: $T(n) = T(k) + T(n-k-1) + O(n)$.
Best case: $T(n) = 2T(n/2) + O(n)$ which gives $O(n \log n)$.
Worst case: $T(n) = T(n-1) + O(n)$ which gives $O(n^2)$.

★ A-95 What is the space complexity?

In-place, but uses recursion stack: **O(log n) on average, O(n) in the worst case**. Recursing on the smaller part first keeps stack at O(log n).

A-96 Is Quick Sort stable? Is it in-place?

It is **in-place** (no extra array) but **not stable** (equal elements may change relative order).

A-97 Why does randomization help?

A bad pivot sequence becomes very unlikely whatever the input. The worst case is still possible in theory, but with **very low probability**, so no adversarial input can reliably slow it down.

A-98 What are Lomuto and Hoare partition schemes?

Lomuto: one scan pointer, pivot at end, simple but more swaps. **Hoare:** two pointers moving towards each other, fewer swaps and usually faster.

A-99 Why is Quick Sort often faster than Merge Sort in practice?

It sorts in place, has small constants and is **cache-friendly**, even though Merge Sort has a better worst case.

A-100 What are the edge cases?

Empty array, one element, all elements equal, already sorted, reverse sorted, many duplicates (use **three-way partitioning** to handle them well).

A-101 How will you compare deterministic and randomized versions in the lab?

Run both on random, sorted and reverse-sorted inputs of increasing size and record time. Deterministic first-pivot becomes slow on sorted input; randomized stays near n log n.

A-102 What are the advantages and disadvantages?

Advantages: fast on average, in-place, widely used.
Disadvantages: $O(n^2)$ worst case, not stable, recursive.

A-103 What output does the program print?

The sorted array (ascending), plus the time taken by each version for the chosen input size.

A-104 Real-life use?

Library sorting routines (for example introsort, a Quick Sort hybrid, in C++ `std::sort`) and general in-memory sorting.

A7(a). Mini Project: Matrix Multiplication and Multithreaded Matrix Multiplication

Threading API depends on the language (Java Thread, C++ std::thread, pthreads, Python). In CPython the GIL stops threads running CPU work in parallel, so use multiprocessing or another language to see real speed-up.

★ A-105 What is the aim of this mini project?

To implement matrix multiplication **sequentially** and with **multiple threads**, then compare execution time.

A-106 When can two matrices be multiplied?

When the **number of columns of A equals the number of rows of B**. If A is m×n and B is n×p, the result C is m×p.

★ **A-107 What is the formula for matrix multiplication?**

C[i][j] = Σ A[i][k] × B[k][j], for k = 1 to n. Each result cell is a dot product of a row of A and a column of B.

★ **A-108 What is the time and space complexity of the standard method?**

For two n×n matrices: time **O(n³)** (three nested loops), space **O(n²)** for the result matrix.

★ **A-109 How is multithreading applied?**

Divide the work: each thread computes **one row (or a block of rows)** of C. Threads run in parallel and the main thread waits (`join`) for all to finish.

A-110 Why is there no race condition in this design?

Each thread **writes to different cells** of C and only reads A and B, so no two threads modify the same memory.

A-111 Will multithreading always be faster?

No. For **small matrices** thread creation and management overhead can exceed the gain. Speed-up is also limited by the number of CPU cores and memory bandwidth.

A-112 What is the ideal parallel running time?

With p processors, ideally about **O(n³ / p)**. Real speed-up is lower because of overhead and the sequential parts (Amdahl's law).

A-113 How many threads should you create?

Usually about the number of CPU cores, or use a **thread pool**. Creating one thread per cell (n² threads) is wasteful.

A-114 What are the edge cases?

Incompatible dimensions (reject with an error), non-square matrices, empty or 1×1 matrices, very large matrices (memory).

A-115 Is there a faster algorithm than O(n³)?

Yes, **Strassen's algorithm** takes about **O(n^2.81)** using 7 recursive multiplications instead of 8. It has overhead, so it helps only for large n.

A-116 How do you verify both methods are correct?

Multiply the same matrices both ways and compare the results element by element; also test with a small matrix that you can calculate by hand.

A-117 Small example?

[1 2; 3 4] × [5 6; 7 8] = [19 22; 43 50]. For example, C[0][0] = 1·5 + 2·7 = 19.

A-118 What does a performance comparison table look like?

Columns: matrix size, sequential time, multithreaded time, speed-up = sequential / multithreaded. Typically larger sizes show better speed-up. **Record your own measured values; do not quote numbers from others.**

A-119 What are the limitations and possible improvements?

Limits: overhead, memory bandwidth, cache misses. Improvements: block (tiled) multiplication, better loop order (i-k-j), thread pools, Strassen's method, GPU libraries.

A-120 Real-life applications?

Computer graphics, machine learning (neural networks), scientific simulation, image processing, and solving equations.

A7(b). Mini Project: Merge Sort and Multithreaded Merge Sort

★ **A-121 What is the aim of this mini project?**

To implement **Merge Sort** and a **multithreaded version**, and compare their performance on different input sizes.

★ **A-122 What is Merge Sort?**

A **divide-and-conquer** algorithm that splits the array into halves, sorts each half recursively, and **merges** the two sorted halves into one sorted array.

★ A-123 Explain the steps of Merge Sort.

1. **Divide:** split array at the middle.
2. **Conquer:** sort left and right halves recursively.
3. **Combine:** merge two sorted halves by repeatedly picking the smaller front element.

A single element is already sorted (base case).

★ A-124 What is the time and space complexity?

Time **O(n log n) in best, average and worst cases**. Space **O(n)** for the temporary merge array. Recurrence: T(n) = 2T(n/2) + O(n).

A-125 Is Merge Sort stable? Is it in-place?

It is **stable** (equal elements keep their order) but **not in-place** because it needs extra O(n) memory.

★ A-126 How is Merge Sort made multithreaded?

The two halves are **independent**, so each half is sorted by a **separate thread**. After both threads finish (`join`), the main thread merges the results.

A-127 Why does multithreading not give perfect speed-up here?

The final **merge step is sequential** (O(n)) and the top-level work cannot be split, so speed-up is limited. Thread overhead also matters.

A-128 How do you avoid creating too many threads?

Create new threads only up to a **depth limit** (such as number of cores) or only when the sub-array is larger than a **threshold**. Below that, use normal recursion.

A-129 Can two threads corrupt data while sorting halves?

No, because each thread works on a **different part** of the array. The merge starts only after both threads finish.

A-130 What are the edge cases?

Empty array, one element, duplicates, already sorted or reverse sorted input, odd number of elements, very large arrays (memory).

A-131 What are the advantages and disadvantages?

Advantages: guaranteed O(n log n), stable, good for linked lists and external sorting, easy to parallelise.

Disadvantages: extra memory, slower than Quick Sort for small in-memory arrays.

A-132 Real-life applications?

Sorting huge files that do not fit in memory (external sort), sorting linked lists, counting inversions, and stable sorting in libraries (for example Python's Timsort is based on merge sort).

A-133 How do you present performance comparison?

Make a table of input size versus sequential time versus multithreaded time and compute speed-up. Use your own measurements; results depend on CPU cores and language.

A-134 How do you check correctness?

Verify the output is in non-decreasing order, has the same elements as the input, and matches the result of the sequential version or the language's built-in sort.

A-135 What improvements could be made?

Parallel merge, thread pool, switching to insertion sort for small parts, and using one shared temporary array to reduce memory allocation.

A7(c). Mini Project: Naive String Matching and Rabin-Karp Algorithm

★ A-136 What is the aim of this mini project?

To implement **naive string matching** and the **Rabin-Karp algorithm** to find all occurrences of a pattern in a text, and compare them.

A-137 What is the string matching problem?

Given a text T of length n and a pattern P of length m, find **all positions** where P appears in T.

★ A-138 Explain the naive algorithm.

Slide the pattern over the text one position at a time. At every position compare characters from left to right; if all m match, report the index.

★ A-139 What is the time complexity of the naive algorithm?

Worst case **O((n − m + 1) × m)**, roughly O(nm), for example text AAAA...A and pattern AAAB. Best case about O(n). Space **O(1)**.

★ A-140 Explain the Rabin-Karp algorithm.

1. Compute a numeric **hash** of the pattern.
2. Compute the hash of the first m characters of the text.
3. Slide the window: if hashes are equal, compare characters to confirm; otherwise move on.
4. Update the next window's hash using a **rolling hash** in O(1).

★ A-141 What is a rolling hash? Write the formula.

A hash updated quickly when the window slides by one character:
t(s+1) = (d × (t(s) − T[s] × h) + T[s+m]) mod q, where d = number of characters in the alphabet (radix), q = a prime modulus, h = d^(m−1) mod q.

A-142 What is a spurious hit?

A position where the **hash matches but the strings do not**, caused by hash collisions. That is why Rabin-Karp always verifies by comparing characters.

★ A-143 What is the time complexity of Rabin-Karp?

Average and best **O(n + m)**; worst case **O(nm)** when there are many spurious hits (for example all characters equal). Space O(1).

A-144 Why do we use a prime number as the modulus?

A good prime spreads hash values evenly, reducing collisions and spurious hits. Using mod also keeps numbers small and avoids overflow.

★ A-145 When is Rabin-Karp better than naive matching?

For **long texts**, and especially for **multiple-pattern search** (compare each window's hash to a set of pattern hashes), as in plagiarism detection.

A-146 When might naive matching be preferred?

For short strings or simple programs: it is easy, needs no hash, and has no collision handling.

A-147 What are the edge cases?

Pattern longer than text (no match), empty pattern, text or pattern with repeated characters, overlapping matches, case sensitivity, negative values after the mod operation (add q to fix).

A-148 What output should the program give?

The starting index (usually **0-based**) of every match, or a message that the pattern is not found. Example: text = ABABCABAB, pattern = ABAB gives indices 0 and 5.

A-149 Which other algorithm gives guaranteed O(n + m)?

KMP (Knuth-Morris-Pratt), which uses a prefix table to avoid re-checking characters. It is often asked as a follow-up.

A-150 What are the applications?

Text editors (find), plagiarism detection, DNA sequence search, search engines, intrusion detection (signature matching).

A-151 What are the limitations of Rabin-Karp?

Worst case is still O(nm), the hash function and modulus must be chosen well, and it needs extra verification work when hashes collide.

A7(d). Mini Project: Exact and Approximation Algorithms for the Travelling Salesperson Problem (TSP)

★ A-152 What is the aim of this mini project?

To solve the **Travelling Salesperson Problem** using an **exact method** and an **approximation method**, and compare cost and running time.

★ A-153 What is TSP?

Given n cities and distances between them, find the **shortest route that visits every city exactly once and returns to the starting city**.

A-154 Is TSP easy or hard?

It is **NP-hard**. No known algorithm solves it exactly in polynomial time for all inputs.

★ A-155 What exact algorithms can be used?

Brute force (try all tours) O((n−1)!), **Dynamic Programming (Held-Karp)** $O(n^2 \cdot 2^n)$ time and $O(n \cdot 2^n)$ space, or **branch and bound** with pruning (worst case exponential).

A-156 Explain brute force TSP.

Fix a start city, generate all (n−1)! orders of the remaining cities, calculate each tour cost, and keep the minimum.

A-157 Explain the idea of Held-Karp DP.

Let C(S, j) be the shortest path that starts at city 1, visits exactly the set S of cities and ends at j. Build answers for bigger sets from smaller ones. Final answer: minimum over j of C(all, j) + dist(j, 1).

★ A-158 Which approximation algorithm is commonly used?

The **MST-based 2-approximation** for metric TSP:

1. Build a **Minimum Spanning Tree** (Prim or Kruskal).
2. Do a **preorder (DFS) walk** of the tree.
3. Skip repeated cities (shortcutting) to get the tour.

★ A-159 What is the approximation ratio of this algorithm? Why?

2 (tour cost ≤ 2 × optimal). Reason: removing one edge from the optimal tour gives a spanning tree, so MST ≤ OPT. The full walk costs 2 × MST, and shortcuts do not add cost when the **triangle inequality** holds.

★ A-160 What is the triangle inequality?

For any three cities, **d(a, c) ≤ d(a, b) + d(b, c)**: going directly is never longer than going through another city. The 2-approximation guarantee needs this condition.

A-161 What is the Christofides algorithm?

A better approximation for metric TSP with ratio **1.5**. It adds a minimum-weight perfect matching on odd-degree vertices of the MST, forms an Euler tour and shortcuts it.

A-162 What is the nearest-neighbour heuristic?

Start at a city and always go to the nearest unvisited city. It is fast ($O(n^2)$) but gives **no guaranteed ratio**.

A-163 Why use approximation algorithms?

Exact algorithms become impractical beyond a few dozen cities. Approximation gives a **good tour quickly** with a known quality bound.

A-164 What are the edge cases?

n = 1 or 2, asymmetric distances (distance A to B differs from B to A), missing edges (incomplete graph), repeated cities, distances that violate the triangle inequality.

A-165 How do you compare the exact and approximate results?

Run both on the same distance matrix and compare tour cost and runtime. The approximate cost is **always ≥ the optimal cost**, and within 2× for metric inputs.

A-166 What output should the program give?

The order of cities in the tour (starting and ending at the same city) and the **total tour cost**.

A-167 What are the applications of TSP?

Delivery and vehicle routing, circuit board drilling, warehouse picking, telescope scheduling and DNA sequencing.

A-168 What is the difference between TSP and the Hamiltonian cycle problem?

A Hamiltonian cycle only asks **whether** a cycle visiting every vertex once exists. TSP asks for the **cheapest** such cycle.

3. Group B: Machine Learning

B0. Machine Learning Basics (Common to All ML Practicals)

These concept questions are asked in almost every ML viva. Implementation is shown generally; Python (pandas, NumPy, scikit-learn, Keras) is the usual language.

★ B-1 What is Machine Learning?

A branch of AI where computers **learn patterns from data** and make predictions or decisions without being explicitly programmed for each rule.

★ B-2 What is the difference between supervised, unsupervised and reinforcement learning?

Supervised: learns from labelled data (input and known output). **Unsupervised:** finds structure in unlabelled data. **Reinforcement:** an agent learns by trial and error using rewards.

B-3 What are features and the target variable?

Features are the input columns used to predict. The **target (label)** is the output column we want to predict, for example fare_amount or Survived.

★ B-4 Why do we split data into training and testing sets?

The model learns on the **training set** and is evaluated on unseen **test data** to check how well it generalises. A common split is 80:20 or 70:30 (`train_test_split`).

★ B-5 What is overfitting and underfitting?

Overfitting: model memorises training data, so training score is high but test score is low.
Underfitting: model is too simple to learn the pattern, so both scores are low.

B-6 How can overfitting be reduced?

Use more data, simpler model, **cross-validation**, regularisation, pruning or limiting depth of trees, dropout and early stopping in neural networks.

B-7 What is data preprocessing? Name the main steps.

Preparing raw data for modelling: handling **missing values**, removing duplicates, treating **outliers**, **encoding** categorical data, **scaling** features and splitting data.

B-8 How do you handle missing values?

Drop rows or columns with too many missing values, or **impute**: fill with mean or median (numeric) or mode (categorical).

B-9 What is an outlier and how is it detected?

A value far from the rest of the data. Detect using a **boxplot / IQR rule** (below Q1 − 1.5·IQR or above Q3 + 1.5·IQR) or **z-score** (beyond about ±3).

B-10 What is encoding? Name two types.

Converting text categories to numbers. **Label encoding** gives each category an integer; **one-hot encoding** makes a separate 0/1 column for each category.

B-11 What is feature scaling and why is it needed?

Bringing features to a similar range. Needed for **distance-based** (KNN, K-Means, SVM) and **gradient-based** (neural network, gradient descent) methods. Tree-based methods do not need it.

B-12 What is cross-validation?

Splitting data into k folds; train on k−1 folds and test on the remaining one, repeated k times, then average. It gives a more reliable estimate than one split.

B-13 What is a hyperparameter?

A setting chosen **before** training, such as k in KNN, number of trees, learning rate or epochs. Parameters (like weights) are learned **during** training.

B-14 Which Python libraries are commonly used?

pandas (data tables), **NumPy** (arrays), **Matplotlib / Seaborn** (plots), **scikit-learn** (ML models and metrics), **TensorFlow / Keras** (neural networks).

B1. Uber Fare Prediction (Linear Regression and Random Forest Regression)

Typical dataset: Uber fares CSV with pickup/dropoff latitude and longitude, pickup datetime, passenger count and fare_amount. Column names can vary in your copy. Do not quote accuracy figures unless you computed them.

★ B-15 What is the aim of this practical?

To **predict the Uber fare** from trip details by preprocessing the data, detecting outliers, checking correlation, and building **Linear Regression** and **Random Forest Regression** models, then comparing them.

★ B-16 What are the features and the target?

Target: **fare_amount**. Features: pickup and dropoff coordinates, passenger count and time-based values (hour, day, month, year) taken from the pickup datetime. A very useful derived feature is the **trip distance**.

B-17 Is this classification or regression? Why?

Regression, because the target (fare) is a continuous numeric value.

★ B-18 What preprocessing steps did you perform?

Dropped irrelevant columns (such as the key / index), removed or filled **null values**, converted pickup_datetime to datetime and extracted hour/day/month/year, computed **distance** from coordinates, removed invalid rows (zero or out-of-range coordinates, negative fare, passenger count 0 or too high).

B-19 How do you calculate distance from latitude and longitude?

Using the **Haversine formula**, which gives great-circle distance on a sphere (Earth radius ≈ 6371 km), instead of plain Euclidean distance on degrees.

★ B-20 How did you detect and handle outliers?

Using **boxplots and the IQR method** (or z-score). Outliers such as negative or extremely high fares and impossible distances were removed or capped because they distort the Linear Regression line.

★ B-21 What is correlation and how do you use it here?

Correlation measures the linear relationship between two variables, from **−1 to +1** (`df.corr()`, shown as a heatmap). Features strongly correlated with fare (like distance) are useful; features strongly correlated with each other are redundant (multicollinearity).

★ B-22 Explain Linear Regression.

It fits a straight line (or plane) **y = b0 + b1·x1 + b2·x2 + ...** by minimising the sum of squared errors. Each coefficient shows how much the fare changes when that feature increases by one unit.

★ B-23 Explain Random Forest Regression.

It builds **many decision trees** on random samples of data (bootstrap) and random subsets of features, and **averages their predictions**. Averaging reduces overfitting and handles non-linear patterns.

B-24 Why compare these two models?

Linear Regression is a simple, interpretable baseline that assumes a linear relation. Random Forest can model complex non-linear relations and is usually more accurate but harder to interpret and slower.

★ B-25 Which evaluation metrics did you use?

R^2 score (how much variance is explained, closer to 1 is better), **RMSE** (error in the same units as fare), and optionally **MAE**. Accuracy is not used for regression.

B-26 What are the formulas for RMSE and R^2?

RMSE = $\sqrt{\Sigma (y - \hat{y})^2 / n}$

$R^2 = 1 - SS_{res} / SS_{tot}$, where $SS_{res} = \Sigma (y - \hat{y})^2$ and $SS_{tot} = \Sigma (y - \bar{y})^2$.

B-27 How do you decide which model is better?

The model with **higher R^2 and lower RMSE on the test set** is better. Compare test scores, not training scores.

B-28 Is feature scaling needed for this practical?

Not for Random Forest. For Linear Regression, plain least squares works without scaling, but scaling helps interpretation and gradient-based solvers.

B-29 Which hyperparameters can you tune in Random Forest?

`n_estimators` (number of trees), `max_depth`, `min_samples_split`, `min_samples_leaf`, `max_features`.

B-30 How can the model be improved?

Better outlier removal, more engineered features (distance, rush-hour flag, airport trips), hyperparameter tuning with cross-validation, and trying other models such as Gradient Boosting.

B2. Email Spam Classification (KNN and SVM)

Typical dataset: a CSV where each row is an email, columns are word counts and the last column is spam / not spam. If your data is raw text, add text-to-number conversion first.

★ B-31 What is the aim of this practical?

To **classify emails as spam or not spam** using **K-Nearest Neighbors (KNN)** and **Support Vector Machine (SVM)** and compare their performance.

B-32 Is this classification or regression?

Binary classification: two classes (spam = 1, not spam = 0).

B-33 What are the features and target?

Features: numeric representation of the email, such as **word frequency counts** (bag-of-words) or TF-IDF values. Target: the spam / not-spam label.

B-34 If the emails are raw text, how do you convert them for the model?

Tokenise, convert to lowercase, remove punctuation and stop words, then convert to numbers using **Bag-of-Words, Count Vectorizer or TF-IDF**.

★ B-35 Explain how KNN works.

For a new email, find the **k closest training emails** (using a distance such as Euclidean) and assign the **majority class** among them. It is a **lazy learner** since it stores data and does the work at prediction time.

★ B-36 How do you choose the value of k?

Try several values and pick the one with best validation score (plot error versus k). Use an **odd k** for two classes to avoid ties. A very small k overfits, a very large k underfits.

★ B-37 Explain how SVM works.

SVM finds the **hyperplane that separates classes with the maximum margin**. The points nearest to the boundary are the **support vectors**. A kernel (linear, RBF) lets it separate non-linear data.

B-38 What do the kernel and the C parameter mean in SVM?

The **kernel** transforms data so it becomes separable. **C** controls the trade-off: a large C tries to classify all training points correctly (risk of overfitting), a small C allows a wider margin with some errors.

★ B-39 Why is scaling important for KNN and SVM?

Both use distances. A feature with large values would dominate. Scaling (standardisation or normalisation) puts features on a similar scale.

★ B-40 Which evaluation metrics do you use and why?

Confusion matrix, accuracy, precision, recall, F1-score. For spam, **precision** matters because a false positive means a genuine email goes to spam. Accuracy alone can mislead if classes are imbalanced.

B-41 What are false positive and false negative in spam detection?

False positive: a real email marked as spam (harmful, may lose important mail). **False negative:** a spam email that reaches the inbox.

B-42 Compare KNN and SVM for this problem.

KNN is simple, needs no training but is slow for large data and sensitive to many features. SVM trains slower but predicts quickly and works well in high-dimensional text data.

B-43 What is the curse of dimensionality?

When there are very many features (thousands of words), distances between points become less meaningful, hurting KNN. Feature selection or dimensionality reduction helps.

B-44 Name another popular spam classifier.

Naive Bayes, which is fast and works very well for text classification.

B-45 How can the spam classifier be improved?

Better text cleaning, TF-IDF, stop-word handling, tuning k / C / kernel, handling class imbalance, and trying Naive Bayes or ensemble models.

B3. Bank Customer Churn Prediction (Neural Network)

Typical dataset: Churn_Modelling.csv with CreditScore, Geography, Gender, Age, Tenure, Balance, NumOfProducts, HasCrCard, IsActiveMember, EstimatedSalary and the target Exited.

★ B-46 What is the aim of this practical?

To build a **neural network** that predicts whether a bank customer will **leave the bank (churn)**.

B-47 What is customer churn?

When a customer stops using a company's services. Predicting it lets the bank take steps to retain customers.

B-48 What are the features and target?

Target: **Exited** (1 = left, 0 = stayed). Features: credit score, geography, gender, age, tenure, balance, number of products, card ownership, active status, salary. Identifier columns (RowNumber, CustomerId, Surname) are dropped.

★ B-49 What preprocessing is needed?

Drop ID/name columns, **encode** Geography and Gender (one-hot / label encoding), **scale** numeric features (StandardScaler), and split into training and test sets.

★ B-50 Why is scaling needed for neural networks?

Features with large values (like Balance) would dominate. Scaling makes training faster and more stable for gradient descent.

★ B-51 Describe the neural network architecture.

An **input layer** (one neuron per feature), one or more **hidden layers** (ReLU activation), and an **output layer** with **1 neuron and sigmoid activation** that gives a probability of churn.

B-52 Which loss function and optimizer are used?

Binary cross-entropy loss (for two classes) and the **Adam** optimizer (an adaptive form of gradient descent).

B-53 What are epochs and batch size?

Epoch: one full pass through the training data. **Batch size:** number of samples processed before the weights are updated.

★ B-54 What are forward propagation and backpropagation?

Forward propagation: inputs pass through layers to produce a prediction. **Backpropagation:** the error is sent backwards to compute gradients and update the weights.

B-55 How is the sigmoid output converted to a class?

Use a **threshold**, usually 0.5: probability above 0.5 means churn (1), otherwise stay (0). The threshold can be changed to favour recall or precision.

B-56 Which evaluation metrics do you use?

Accuracy, confusion matrix, precision, recall, F1-score. Churn data is often imbalanced, so recall and F1 are important along with accuracy.

B-57 How can overfitting be controlled in the network?

Dropout, **early stopping**, fewer neurons or layers, more data, and checking validation loss during training.

B-58 What is the role of an activation function?

It adds **non-linearity** so the network can learn complex patterns. Examples: ReLU (hidden layers), sigmoid (binary output), softmax (multi-class output).

B-59 Which libraries are used to build the network?

TensorFlow / Keras (Sequential model with Dense layers) or PyTorch; scikit-learn for preprocessing and metrics.

B-60 How can the bank use the result?

Identify customers likely to leave and offer targeted incentives or service improvements.

B4. Gradient Descent to Find the Local Minimum of a Function

A commonly used example is f(x) = (x + 3)², with minimum at x = −3. Your function may differ.

★ B-61 What is the aim of this practical?

To use **Gradient Descent** to find a **local minimum** of a given function by repeatedly moving in the direction of decreasing slope.

★ B-62 What is Gradient Descent?

An iterative optimisation algorithm that updates a variable in the **opposite direction of the gradient (slope)** to reduce the function value.

★ B-63 Write the update rule.

x_new = x_old − α × f′(x_old), where α (alpha) is the **learning rate** and f′(x) is the derivative (gradient) at x.

B-64 Why do we subtract the gradient?

The gradient points to the steepest **increase**. To go downhill (towards a minimum) we move in the opposite direction.

★ B-65 What is the learning rate? What if it is too large or too small?

It is the step size. **Too large:** steps overshoot and may diverge. **Too small:** very slow convergence. A value like 0.01 to 0.1 is typical for simple functions.

B-66 When do you stop the algorithm?

When the **change in x is smaller than a tolerance** (for example 0.000001), when the gradient is almost zero, or after a **maximum number of iterations**.

★ B-67 Show a hand-calculated example.

f(x) = (x + 3)², f′(x) = 2(x + 3), start x = 2, α = 0.1.
Step 1: x = 2 − 0.1 × 10 = 1.0
Step 2: x = 1 − 0.1 × 8 = 0.2
x keeps moving towards **−3**.

B-68 What is the difference between local minimum and global minimum?

A **local minimum** is the lowest point in a neighbourhood. The **global minimum** is the lowest point of the whole function. Gradient descent may get stuck in a local minimum for non-convex functions.

B-69 What is a convex function?

A function with a single bowl shape, where any local minimum is also the global minimum. Gradient descent reaches it with a suitable learning rate.

B-70 What are the types of gradient descent?

Batch (uses all data per step), **Stochastic (SGD)** (one sample per step), **Mini-batch** (small groups of samples).

B-71 Where is gradient descent used in ML?

To minimise the **loss function** while training Linear Regression, Logistic Regression and neural networks.

B-72 What problems can occur and how are they fixed?

Divergence (reduce α), slow progress (increase α or use momentum / Adam), stuck at a saddle point or local minimum (try different starting points).

B5. KNN Classification on the Diabetes Dataset

Typical dataset: Pima Indians Diabetes dataset (768 rows) with Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age and target Outcome.

★ B-73 What is the aim of this practical?

To classify whether a patient is **diabetic or not** using the **K-Nearest Neighbors** algorithm and evaluate it with a **confusion matrix, accuracy, error rate, precision and recall**.

B-74 What are the features and target in this dataset?

Features: Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age. Target: **Outcome** (1 = diabetic, 0 = not diabetic).

★ B-75 What data cleaning is needed?

In columns like **Glucose, BloodPressure, SkinThickness, Insulin and BMI, a value of 0 is not medically possible**, so treat zeros as missing and replace them with the column **median (or mean)**.

★ B-76 Why do you scale the data for KNN?

KNN uses distance, and features such as Insulin have much bigger values than others. Scaling stops large-valued features from dominating.

★ B-77 How does KNN classify a new patient?

It calculates the distance from the new patient to all training patients, picks the **k nearest**, and assigns the **majority class**.

B-78 Which distance measures can be used?

Euclidean ($\sqrt{\Sigma(x - y)^2}$), **Manhattan** ($\Sigma|x - y|$), and Minkowski (general form). Euclidean is the default.

★ B-79 How did you choose k?

By trying different k values and plotting **error rate versus k** on test or validation data, then choosing the k with the lowest error. Odd values avoid ties.

★ B-80 Explain the confusion matrix.

A 2×2 table of **TP** (diabetic predicted diabetic), **TN** (non-diabetic predicted non-diabetic), **FP** (non-diabetic predicted diabetic) and **FN** (diabetic predicted non-diabetic).

B-81 Write the formulas for accuracy, error rate, precision and recall.

Accuracy = (TP + TN) / (TP + TN + FP + FN)
Error rate = 1 − Accuracy
Precision = TP / (TP + FP)
Recall = TP / (TP + FN)

★ B-82 In a medical problem like this, which metric is most important?

Recall (sensitivity), because a false negative means a diabetic patient is missed and untreated. Precision still matters to avoid unnecessary worry and tests.

B-83 What are the disadvantages of KNN?

Slow prediction on large data, needs scaling, sensitive to irrelevant features and outliers, and performance drops with many features.

B-84 How can the KNN model be improved?

Scale features, handle zeros and outliers, select useful features, tune k and distance metric, use weighted KNN, and use cross-validation.

B6. K-Means and Hierarchical Clustering (Elbow Method)

Typical dataset: a customer or sales dataset with numeric columns. Clustering is unsupervised, so there is no target column.

★ B-85 What is the aim of this practical?

To group similar data points into clusters using **K-Means** and **Hierarchical clustering**, and use the **elbow method** to select the number of clusters.

B-86 What is clustering? Is it supervised?

Grouping data so that points in the same cluster are more similar than those in other clusters. It is **unsupervised learning** (no labels).

★ B-87 Explain the K-Means algorithm.

1. Choose k.
2. Pick k initial **centroids**.
3. Assign every point to the **nearest centroid**.
4. Recalculate each centroid as the **mean** of its points.
5. Repeat steps 3-4 until assignments stop changing.

★ B-88 What is the objective of K-Means?

To minimise **WCSS (Within-Cluster Sum of Squares)**, also called **inertia**: the sum of squared distances between each point and its cluster centroid.

★ B-89 What is the elbow method?

Run K-Means for k = 1, 2, 3, ... and plot **WCSS versus k**. WCSS always falls as k increases; the point where the curve bends like an elbow, after which the fall is small, is chosen as k.

B-90 What is the silhouette score?

A measure from −1 to +1 of how well a point fits its own cluster compared with the nearest other cluster. Higher is better. It is another way to select k.

B-91 Why is feature scaling needed?

K-Means uses distance, so features with larger ranges dominate. Standardise features before clustering.

B-92 What are the limitations of K-Means?

Needs k in advance, depends on initial centroids (use **k-means++**), sensitive to outliers, and prefers round, similar-sized clusters.

★ B-93 Explain Hierarchical clustering.

It builds a **tree of clusters**. **Agglomerative** (bottom-up) starts with each point as its own cluster and repeatedly merges the two closest clusters. **Divisive** (top-down) starts with one cluster and splits it.

★ B-94 What is a dendrogram?

A tree diagram showing the order and distance at which clusters were merged. **Cutting it at a chosen height** gives the final clusters.

B-95 What are linkage methods?

Rules to measure distance between clusters: **single** (closest points), **complete** (farthest points), **average**, and **Ward** (minimises increase in variance).

B-96 How do you evaluate clustering without labels?

Use WCSS, silhouette score, and visual inspection of cluster plots and cluster summaries. There is no accuracy because there is no true label.

B-97 Where is clustering used?

Customer segmentation, market basket grouping, image compression, document grouping and anomaly detection.

B7. Stock Market Analysis and Prediction

Typical data: historical daily prices (Open, High, Low, Close, Adj Close, Volume) from a source such as Yahoo Finance. Predictions are for learning only and are not financial advice.

★ B-98 What is the aim of this practical?

To **analyse historical stock data** and build a model that **predicts future prices** (usually the closing price).

B-99 What type of data is stock data?

Time-series data: values recorded at regular time steps where order matters.

B-100 What do Open, High, Low, Close and Volume mean?

Open: first trade price of the day. High / Low: highest / lowest price. **Close:** last trade price. Volume: number of shares traded.

★ **B-101 Which analysis techniques did you use?**

Trend plots, **moving averages (for example 50-day and 200-day)**, daily returns, volatility and correlation between stocks or features.

B-102 What is a moving average?

The average of the last n closing prices, recalculated each day. It smooths short-term noise and shows the trend.

★ **B-103 Which models can predict stock prices?**

Linear Regression (simple baseline), **ARIMA** (statistical time series model), and **LSTM (Long Short-Term Memory)** neural networks.

★ **B-104 Why is LSTM suitable?**

LSTM is a recurrent neural network that **remembers information over long sequences**, so it can learn from past price patterns.

★ **B-105 Why must the data be split in time order, not randomly?**

Random splitting would let the model **see the future** during training (data leakage). Train on earlier dates and test on later dates.

B-106 Why is scaling (MinMaxScaler) used for LSTM?

Neural networks train better when inputs are in a small range such as 0 to 1.

B-107 What is a look-back window / time step?

The number of previous days (for example 60) used as input to predict the next day's price.

B-108 Which metrics are used to evaluate?

RMSE and **MAE** (average error), and sometimes R^2. Plot **actual versus predicted** prices for visual checking.

B-109 Can the model predict the market perfectly?

No. Prices depend on news, events and investor behaviour. The model finds patterns in past data only, so results should be treated as **estimates**.

B-110 How can the model be improved?

Add more features (volume, indicators, sentiment), tune the window size and layers, use more data, and test across different periods.

B8. Titanic Survival Prediction

Typical dataset: Titanic train.csv (891 passengers) with PassengerId, Survived, Pclass, Name, Sex, Age, SibSp, Parch, Ticket, Fare, Cabin, Embarked.

★ **B-111 What is the aim of this practical?**

To predict whether a **passenger survived** the Titanic using features such as class, sex and age with a **classification** model.

B-112 What is the target variable and which features are used?

Target: **Survived** (1 = survived, 0 = did not). Common features: **Pclass, Sex, Age, SibSp, Parch, Fare, Embarked**.

★ **B-113 Which columns have missing values and how do you handle them?**

In the standard training file, **Age** (about 177 missing), **Cabin** (about 687 missing) and **Embarked** (2 missing). Fill Age with the median, Embarked with the mode, and **drop Cabin** because most values are missing.

B-114 Which columns are usually dropped and why?

PassengerId, Name, Ticket (identifiers with little predictive value) and **Cabin** (too many missing values). Name may be used to extract a title such as Mr or Mrs.

★ **B-115 How do you encode categorical columns?**

Convert **Sex** to 0/1 and **Embarked** using label or one-hot encoding, since models need numeric input.

B-116 What is feature engineering? Give an example.

Creating new useful features. Example: **FamilySize = SibSp + Parch + 1**, or extracting the title from Name.

★ **B-117 Which models can be used?**

Logistic Regression, Decision Tree, **Random Forest**, KNN or SVM. All are classification algorithms.

★ **B-118 Explain Logistic Regression.**

Despite the name it is a classifier. It applies the **sigmoid function** σ(z) = 1 / (1 + e^(−z)) to a linear combination of features to give a probability between 0 and 1.

B-119 Which evaluation metrics do you use?

Accuracy, confusion matrix, precision, recall and F1-score, on the test set.

B-120 What is data leakage and how do you avoid it here?

Using information from test data during training. Avoid it by **fitting imputers and scalers only on training data** and then applying them to test data.

B-121 What patterns are usually seen in this dataset?

Typically, **women and first-class passengers had higher survival rates**. Always confirm using your own exploratory data analysis (EDA) plots, not memory.

B-122 How can you improve the model?

Better imputation of Age, more engineered features, hyperparameter tuning, cross-validation and using ensemble models.

B9. Signature Identification Using a Custom Dataset

Implementation varies: students often use a CNN (Convolutional Neural Network) on signature images, while some use classical features with SVM or KNN. Answers cover the usual design.

★ **B-123 What is the aim of this practical?**

To build a model that **identifies the person who made a signature** (or checks whether it is genuine) from signature images in a **custom dataset** collected by the student.

★ **B-124 What is the difference between signature identification and verification?**

Identification: decide **who** signed among several people (multi-class). **Verification:** decide whether a signature is **genuine or forged** for a claimed person (binary).

★ **B-125 How did you create the custom dataset?**

Collected several signatures from each person on paper, scanned or photographed them, and saved images in **one folder per person (class)**. Genuine and forged samples can be kept in separate folders for verification.

★ **B-126 What image preprocessing is done?**

Convert to **grayscale**, **resize** to a fixed size (for example 128×128), remove noise, **binarise** (thresholding), and **normalise** pixel values to the 0-1 range.

B-127 What is data augmentation and why is it used?

Creating extra training images by small rotations, shifts or zoom. It is useful because a custom dataset is small and augmentation **reduces overfitting**.

★ **B-128 Why is a CNN suitable for images?**

A CNN automatically learns visual features like strokes and curves using **convolution filters**, and its pooling layers keep the important information while reducing size.

B-129 Describe a simple CNN architecture.

Input image → **Convolution + ReLU** → **Max Pooling** (repeated) → **Flatten** → **Dense layers** → **Softmax** output (one neuron per person) or sigmoid for genuine/forged.

B-130 What are convolution and pooling layers?

Convolution slides small filters over the image to detect patterns. **Pooling** (for example max pooling) shrinks the feature map by keeping the strongest values.

B-131 Which loss function is used?

Categorical cross-entropy for multiple persons, or **binary cross-entropy** for genuine versus forged.

★ **B-132 Which metrics are used to evaluate?**

Accuracy and confusion matrix. For verification, **FAR (False Acceptance Rate)**: forged accepted as genuine, and **FRR (False Rejection Rate)**: genuine rejected.

B-133 What challenges are there in signature recognition?

A person's signature varies each time, **skilled forgeries** look very similar, scan quality and pen differences, and a small dataset.

B-134 How can the model be improved?

More samples per person, augmentation, transfer learning with a pre-trained network, better preprocessing, and testing on unseen signatures.

B-135 Which libraries are used?

OpenCV / PIL for image handling, **NumPy**, **TensorFlow / Keras** (or PyTorch) for the model, scikit-learn for metrics and splitting.

B-136 Are there privacy concerns?

Yes. Signatures are **biometric / personal data**, so collect them with **consent**, store them securely and use them only for the stated purpose.

4. Group C: Blockchain Technology

C0. Blockchain Fundamentals (Common to All Blockchain Practicals)

Fees, tools and test networks change over time. Always state which network (local VM, test network or mainnet) you used.

★ C-1 What is a blockchain?

A **distributed digital ledger** where records are grouped into **blocks**, each block is linked to the previous one by a **hash**, and copies are held by many computers (nodes). Data is very hard to change once added.

★ C-2 What does a block contain?

Transactions (data), a **timestamp**, the **hash of the previous block**, its own **hash**, and a **nonce** (used in Proof of Work). Ethereum blocks also have a gas limit and gas used.

★ C-3 What is a hash and why is it important?

A hash (for example SHA-256 in Bitcoin) is a fixed-length fingerprint of data. Changing even one character changes the hash completely, so tampering is easily detected and the chain breaks.

C-4 How does blockchain make data tamper-resistant?

Each block stores the previous block's hash. Changing an old block changes its hash, which breaks every later link; the network's copies would also disagree.

C-5 What is a node?

Any computer that takes part in the blockchain network, keeps a copy of the ledger and validates transactions and blocks.

★ C-6 What is a consensus mechanism?

The rule by which all nodes **agree on the valid state** of the ledger without a central authority. Examples: **Proof of Work (PoW)** and **Proof of Stake (PoS)**.

C-7 What is a distributed ledger?

A ledger shared and synchronised across many nodes, so no single party owns or controls the records.

C-8 What is a smart contract?

A **program stored on the blockchain** that runs automatically when its conditions are met. Once deployed, its code cannot be changed (in the basic design).

C-9 What is the 51% attack?

If one party controls more than half of the network's mining or staking power, it could reverse recent transactions or block others. Large public chains make this very expensive.

C-10 What is a digital signature in blockchain?

A transaction is signed with the sender's **private key**; anyone can verify with the **public key** that it really came from the owner and was not altered.

C-11 What is a Merkle tree?

A tree of hashes where transaction hashes are combined pairwise up to a single **Merkle root** stored in the block header, allowing quick verification of a transaction.

C-12 What are the limitations of blockchain?

Limited **scalability** (low transactions per second), cost (gas fees), energy use in PoW, data being permanent (hard to delete), and privacy concerns on public chains.

C1. Installing MetaMask and Studying Ether Spent per Transaction

*Use a **test network** (for example Sepolia) with free test Ether from a faucet. Test Ether has no real money value. Goerli test network is deprecated. Check the current faucet and network names before the practical.*

★ C-13 What is MetaMask?

A **cryptocurrency wallet** and gateway to Ethereum applications, available as a **browser extension** (Chrome, Firefox, Edge, Brave) and a mobile app. It stores your keys and signs transactions.

★ C-14 What is the aim of this practical?

To install MetaMask, make test transactions, and **study how much Ether is spent per transaction** (the transaction fee) using wallet details and a block explorer such as **Etherscan**.

★ C-15 How is the Ether spent per transaction calculated?

Transaction fee = Gas used × Gas price (effective gas price). After EIP-1559: fee = gas used × (base fee + priority fee). A plain ETH transfer uses **21,000 gas**.

C-16 What units of Ether exist?

1 ETH = 10^18 wei. **1 gwei = 10^9 wei** (10^−9 ETH). Gas prices are quoted in gwei.

C-17 Where can you see the gas used and fee of a transaction?

In MetaMask's **Activity** tab, or by pasting the **transaction hash** on **Etherscan** (test network version), which shows gas used, gas price and transaction fee.

C-18 Why is the fee different for different transactions?

Fees depend on **how complex the operation is** (gas used) and **network congestion** (gas price). Simple transfers use less gas than contract calls.

C-19 Why do we use a test network instead of mainnet?

To practise safely without spending real money. Mainnet transactions cost real Ether.

C2. Creating a Cryptocurrency Wallet Using MetaMask

★ C-20 What are the steps to create a wallet in MetaMask?

1. Install the extension and click **Create a new wallet**.
2. Set a password.
3. Write down the **Secret Recovery Phrase** (12 words) offline and confirm it.
4. The wallet and first **account address** are created.
5. Switch to a test network to get test Ether.

★ C-21 What is a seed phrase (Secret Recovery Phrase)?

A list of 12 (or 24) words that can **regenerate all your private keys and accounts**. Anyone with it controls your funds, so never share it or store it online.

★ C-22 What are a public key, private key and wallet address?

The **private key** is secret and signs transactions. The **public key** is derived from it. The **address** (starts with 0x, 42 characters) is derived from the public key and is shared to receive funds.

C-23 Does a wallet store your coins?

No. Coins stay on the blockchain. The wallet stores your **keys** that prove ownership and let you sign transactions.

C-24 What happens if you lose the seed phrase or password?

Password loss can be fixed by restoring from the seed phrase. If the **seed phrase is lost** and the device is lost, funds **cannot be recovered**, as no company or bank holds a copy.

C3. Bank Account Smart Contract (Deposit, Withdraw, Balance)

Written in Solidity, compiled and deployed using Remix IDE with either the Remix VM (local simulation) or MetaMask on a test network.

★ C-25 What is the aim of this practical?

To write a **Solidity smart contract** that behaves like a bank account with **deposit, withdraw and check-balance** functions, and **deploy** it using Remix IDE.

★ C-26 What are the main functions in the contract?

deposit() is `payable` and adds the sent Ether to the balance. **withdraw(amount)** checks that balance is enough using `require`, reduces the balance and sends Ether. **getBalance()** is a `view` function that returns the balance.

★ C-27 What do `payable`, `msg.sender`, `msg.value` and `require` mean?

`payable`: function can receive Ether. `msg.sender`: address calling the function. `msg.value`: Ether (in wei) sent with the call. `require(cond, msg)`: reverts the transaction if the condition is false.

C-28 What is `view` and why are view calls free?

A `view` function only **reads** blockchain state and does not change it, so calling it from outside costs **no gas**. Functions that change state need a transaction and gas.

★ C-29 How do you compile and deploy the contract in Remix?

Write the code in Remix, choose a compiler version matching `pragma solidity`, click **Compile**, open **Deploy & Run**, select the environment (Remix VM or **Injected Provider - MetaMask**), and click **Deploy**. Then test the functions from the deployed contract panel.

C-30 What security issue should a withdraw function consider?

Reentrancy attack. Follow the **checks-effects-interactions** pattern: check conditions, **update the balance first**, then send Ether.

C4. Solidity Student Data Program (Structure, Array, Fallback) and Gas Analysis

★ C-31 What is the aim of this practical?

To write a Solidity program that stores **student data using a structure and an array**, includes a **fallback function**, deploy it on an Ethereum network, and **analyse gas and transaction fees**.

★ C-32 What is a structure (struct) in Solidity?

A user-defined type that groups different fields. Example: `struct Student { uint id; string name; uint marks; }`.

★ C-33 How are student records stored and read?

Use an array such as `Student[] public students;` Add records with a function that does `students.push(Student(id, name, marks))`, and read them by index or return the array length.

★ C-34 What is a fallback function?

A special function that runs when the contract is called with a **function name that does not exist** or with plain Ether and no matching function. Written as `fallback() external payable`. A separate `receive() external payable` handles plain Ether transfers with empty data.

C-35 Why use a fallback function?

To accept Ether sent directly to the contract, or to handle unknown calls safely (for example by reverting or logging an event).

C-36 What is the difference between `storage` and `memory`?

`storage` data is saved permanently on the blockchain (costly). `memory` data exists only while a function runs (cheaper). `calldata` is read-only function input.

★ C-37 What is gas and what determines it?

Gas measures the **computation and storage work** needed. Writing to storage (such as adding a student) uses more gas than reading. Longer strings and larger arrays cost more.

★ C-38 How do you analyse gas and transaction fee of this program?

After deploying, note **gas used** for deployment and for each function call from the Remix console or Etherscan. **Fee = gas used × gas price**. Compare `addStudent` calls and notice that read-only calls cost nothing.

C-39 What is the gas limit and what is 'out of gas'?

The **gas limit** is the maximum gas you allow for a transaction. If the work needs more, the transaction fails with **out of gas** and the fee spent is not refunded.

C5. Types of Blockchains and Real-World Use Cases

★ C-40 What are the types of blockchain?

Public (anyone can join, e.g. Bitcoin, Ethereum), **Private** (one organisation controls access), **Consortium** (a group of organisations jointly control it), and **Hybrid** (mix of public and private parts).

C-41 What is the difference between permissioned and permissionless blockchains?

Permissionless: anyone can join and validate (public). **Permissioned:** only approved participants can join (private and consortium).

★ C-42 Give real-world use cases of blockchain.

Finance (payments, cryptocurrencies), **supply chain** tracking (IBM Food Trust built on Hyperledger Fabric), **healthcare records**, **voting**, **land records**, **digital identity** and certificate verification.

C-43 Which blockchain type suits a bank consortium or supply chain?

A **consortium (permissioned)** blockchain, as the participants are known, need privacy and faster, controlled transactions.

C6. Creating a Business Network Using Hyperledger

*The older Hyperledger Composer toolkit is **deprecated**; current practice uses **Hyperledger Fabric**. Mention which one your lab used.*

★ C-44 What is Hyperledger?

An **open-source umbrella project hosted by the Linux Foundation** for enterprise blockchains. **Hyperledger Fabric** is its most used framework, a **permissioned** blockchain.

★ C-45 What is a business network in Hyperledger?

A network of organisations that share a ledger and rules (smart contracts) to do business, for example a supplier, manufacturer and retailer tracking goods.

★ C-46 Explain the main components of Hyperledger Fabric.

Peers (hold ledger and run chaincode), **Orderer** (orders transactions into blocks), **Channels** (private sub-networks), **Chaincode** (smart contract), **MSP / Certificate Authority** (identities), **Organisations** (members).

★ C-47 What is chaincode?

Hyperledger's name for a **smart contract**. It is written in Go, JavaScript (Node.js) or Java and defines how assets are created and changed.

C-48 Describe the transaction flow in Fabric.

Client sends a proposal → **endorsing peers** simulate and sign it → client sends it to the **ordering service** → orderer puts it in a block → peers **validate and commit** the block to the ledger.

C-49 What did Hyperledger Composer use for a business network?

A business network archive (.bna) containing a **model file (.cto)**, **script file (.js)**, **access control file (.acl)** and optionally a query file. Composer is no longer maintained.

C7. Blockchain-Based E-Voting dApp

Implementation varies by student. Answers describe a typical Ethereum design.

★ C-50 What is a dApp?

A **decentralised application**: a user interface (HTML/React) connected through **Web3.js or Ethers.js** and a wallet like MetaMask to **smart contracts** running on a blockchain.

★ C-51 How does the e-voting dApp work?

The owner creates candidates in the contract. A registered voter connects MetaMask and calls `vote(candidateId)`. The contract checks the voter has **not voted already**, records the vote and updates the count. Anyone can read the results.

★ C-52 How do you prevent double voting?

Use `mapping(address => bool) hasVoted` and `require(!hasVoted[msg.sender])`, then set it to true after voting. Voters must also be **registered or verified** by the owner.

C-53 What are the benefits and limitations of blockchain voting?

Benefits: transparency, tamper-resistance and auditability. **Limitations:** voter identity verification, vote **privacy** (addresses are pseudonymous, data public), gas cost, scalability, and the risk of coercion or fake identities.

C-54 What security and privacy points should be mentioned?

Restrict admin functions with an **owner-only modifier**, verify eligibility off-chain, and note that real elections would need stronger privacy such as **zero-knowledge proofs** and legal approval.

C8. Blockchain-Based Transparent Charity Application

★ C-55 What problem does a blockchain charity application solve?

Donors often cannot see how funds are used. On blockchain, **every donation and spending record is public and tamper-proof**, increasing trust.

★ C-56 How does the application work?

Donors send Ether using a `payable donate()` function. The contract records donor and amount, keeps a total, and releases funds to a beneficiary only through a recorded request that the owner (or donors) approves.

C-57 How can funds be released safely?

Use **milestone-based release** or **approval voting** by donors, and emit **events** for every withdrawal so anyone can audit them on a block explorer.

C-58 What are the limitations?

The blockchain cannot confirm that money was spent as claimed in the real world (needs off-chain proof), a fake charity can still register, gas fees reduce donations, and Ether's price is volatile.

C9. Blockchain-Based Medical Records Application

★ C-59 Why is blockchain suggested for medical records?

It gives a **secure, tamper-proof history** of records and **patient-controlled access** across hospitals, reducing duplicate tests and record loss.

★ C-60 Should medical data be stored directly on the blockchain? Why?

No. Public chains are costly and visible to everyone. Store **encrypted records off-chain** (database or IPFS) and keep only a **hash and access permissions** on the blockchain.

C-61 What is IPFS?

InterPlanetary File System: a distributed file storage network where files are addressed by the **hash of their content**, often used with blockchain for large files.

★ C-62 How is access control implemented?

The smart contract keeps a mapping of which doctors or hospitals the **patient has permitted**. Only those addresses can fetch the record pointer, and every access can be logged as an event.

C-63 What are the privacy and legal challenges?

Blockchain data is permanent, which conflicts with **right-to-erase** rules in laws like **GDPR** and India's **Digital Personal Data Protection Act, 2023**. Using a **permissioned chain**, off-chain storage and encryption helps.

C10. Blockchain-Based Mental Health Application

This is a sensitive area. The app is a support tool and does not replace professional care or emergency help.

★ C-64 What is the idea of this application?

A platform where users can keep **private mood or therapy records**, give controlled access to therapists, and book sessions, with privacy and audit protection from blockchain.

★ C-65 What is stored on-chain and what off-chain?

On-chain: **hashes of records, consent permissions, therapist credential verification and payment records**. Off-chain and encrypted: journals, chat and session notes.

★ C-66 Why is privacy especially important here?

Mental health data is highly sensitive and carries stigma. Use **wallet-based pseudonymous identity**, **encryption**, minimal data collection and user-controlled **consent** that can be revoked.

C-67 What are the limitations and risks?

On-chain data cannot be erased, losing a private key can lock the user out, and the app cannot handle emergencies, so it must show **crisis helpline information** and advise professional help.

5. Algorithm and Model Comparison Tables

Group A: DAA comparisons

Recursive vs Non-Recursive Fibonacci

Point	Recursive	Non-recursive
Method	Function calls itself	Loop with two variables
Time	**O(2^n)** (exponential)	**O(n)**
Space	O(n) call stack	**O(1)**
Speed	Very slow for large n	Fast
Risk	Stack overflow	None

Greedy vs Dynamic Programming

Point	Greedy	Dynamic Programming
Idea	Take the best local choice at each step	Solve all subproblems and combine, storing results
Optimality	Optimal only if greedy-choice property holds	Always optimal if substructure holds
Looks back?	No, choices are never reconsidered	Yes, uses a table of earlier results
Speed / memory	Usually faster, less memory	Slower, needs a table
Examples	Huffman, Fractional Knapsack	0/1 Knapsack, Fibonacci (DP)

Fractional Knapsack vs 0/1 Knapsack

Point	Fractional	0/1
Items	Can be broken into parts	Take whole or nothing
Method	Greedy by v/w ratio	DP or Branch and Bound
Time	O(n log n)	O(nW) DP; O(2^n) worst B&B
Greedy optimal?	Yes	No

Deterministic vs Randomized Quick Sort

Point	Deterministic	Randomized
Pivot	Fixed rule (first, last, middle)	Chosen at random
Worst case	$O(n^2)$, on sorted input with first/last pivot	$O(n^2)$ still possible but very unlikely
Expected time	O(n log n) on random input only	**O(n log n)** on every input
Behaviour	Same input gives the same run	Run may differ each time

Quick Sort vs Merge Sort

Point	Quick Sort	Merge Sort
Worst time	$O(n^2)$	**O(n log n)**
Average time	O(n log n)	O(n log n)
Extra space	O(log n) stack	**O(n)**
Stable?	No	**Yes**
In-place?	Yes	No
Best for	Arrays in memory (cache-friendly)	Linked lists, external sorting

Sequential vs Multithreaded Matrix Multiplication

Point	Sequential	Multithreaded
Execution	One thread computes all cells	Threads compute rows or blocks in parallel
Time	$O(n^3)$	About $O(n^3 / p)$ ideal, with p cores
Overhead	None	Thread creation and joining
Best for	Small matrices	Large matrices on multi-core CPUs

Sequential vs Multithreaded Merge Sort

Point	Sequential	Multithreaded
Halves sorted	One after another	In parallel by separate threads
Merge step	Sequential	Still sequential, which limits speed-up
Time	O(n log n)	Faster in practice on multi-core, same total work
Concern	None	Thread overhead; limit threads using depth/threshold

Naive String Matching vs Rabin-Karp

Point	Naive	Rabin-Karp
Idea	Compare pattern at every shift	Compare hash values, verify on a match
Worst time	O(nm)	O(nm)
Average / best	O(n) to O(nm)	**O(n + m)**
Extra concept	None	Rolling hash, modulus, spurious hits
Multiple patterns	Slow	Handled well

Exact vs Approximation Algorithms for TSP

Point	Exact (brute force / Held-Karp)	Approximation (MST based)
Result	Optimal tour	Tour at most 2× optimal (metric TSP)
Time	$O((n-1)!)$ or $O(n^2 \cdot 2^n)$	About $O(n^2)$ (MST plus walk)
Practical size	Small n (about 20-25 for DP)	Large n
Needs triangle inequality?	No	Yes, for the guarantee

Backtracking vs Branch and Bound

Point	Backtracking	Branch and Bound
Used for	Finding solutions (feasibility), e.g. N-Queens	Optimisation problems, e.g. 0/1 Knapsack, TSP
Search style	Depth-first	Any order (DFS, BFS or best-first)
Pruning basis	Constraint violation	Bound compared with the best solution so far
Uses a bound?	No	Yes

Group B: Machine Learning comparisons

Supervised vs Unsupervised Learning

Point	Supervised	Unsupervised
Data	Labelled	Unlabelled
Goal	Predict output	Find hidden groups or structure
Examples	Linear Regression, KNN, SVM	K-Means, Hierarchical clustering

Classification vs Regression

Point	Classification	Regression
Output	Category / class	Continuous number
Example	Spam or not spam, Titanic survival	Uber fare, stock price
Metrics	Accuracy, precision, recall, F1	R^2, RMSE, MAE

Linear Regression vs Random Forest Regression

Point	Linear Regression	Random Forest Regression
Relationship	Assumes linear	Handles non-linear
Interpretability	High (coefficients)	Lower
Outliers	Sensitive	More robust
Scaling needed?	Helpful	Not needed
Overfitting	Low (may underfit)	Reduced by averaging, but can overfit

KNN vs SVM

Point	KNN	SVM
Idea	Majority vote of k nearest points	Maximum-margin hyperplane
Training	None (lazy)	Required
Prediction speed	Slow on large data	Fast
Key settings	k, distance	Kernel, C
Scaling	Needed	Needed

K-Means vs Hierarchical Clustering

Point	K-Means	Hierarchical
Number of clusters	Must be given (k)	Chosen after viewing the dendrogram
Output	Flat clusters	Tree (dendrogram)
Speed	Fast, scales to large data	Slow on large data
Selection tool	Elbow method	Dendrogram cut

Training Data vs Testing Data

Point	Training data	Testing data
Purpose	Model learns from it	Model is evaluated on it
Seen by model?	Yes	No
Typical share	70-80%	20-30%

Overfitting vs Underfitting

Point	Overfitting	Underfitting
Training score	High	Low
Test score	Low	Low
Cause	Model too complex	Model too simple
Fix	Regularise, more data, simplify	More features, complex model, train longer

Accuracy vs Precision vs Recall vs F1-score

Metric	Formula	Meaning
Accuracy	(TP + TN) / total	Overall fraction correct
Precision	TP / (TP + FP)	Of predicted positives, how many were right
Recall	TP / (TP + FN)	Of actual positives, how many were found
F1-score	2 × P × R / (P + R)	Balance of precision and recall

Neural Networks vs Traditional ML Algorithms

Point	Neural networks	Traditional ML
Data needed	Large	Small to medium
Feature engineering	Learns features automatically	Often manual
Interpretability	Low (black box)	Higher for many models
Compute	High (GPU often)	Low

Normalization vs Standardization

Point	Normalization (Min-Max)	Standardization (Z-score)
Formula	(x − min) / (max − min)	(x − mean) / std
Range	0 to 1	Mean 0, std 1 (no fixed range)
Outliers	Sensitive	Less sensitive
Used for	Neural nets, image data	KNN, SVM, regression

Group C: Blockchain comparisons

Public vs Private vs Consortium Blockchain

Point	Public	Private	Consortium
Access	Anyone	One organisation	Group of organisations
Speed	Slower	Fast	Fast
Control	Decentralised	Centralised	Shared
Example	Bitcoin, Ethereum	Internal company ledger	Supply chain network

Centralized vs Decentralized Systems

Point	Centralized	Decentralized
Control	One authority	Shared by many nodes
Failure	Single point of failure	More fault tolerant
Speed	Usually faster	Slower
Example	Bank database	Bitcoin network

Cryptocurrency Wallet vs Exchange

Point	Wallet	Exchange
Purpose	Store keys and sign transactions	Buy, sell and trade crypto
Keys held by	You (non-custodial wallet)	The exchange (custodial)
Example	MetaMask	Binance, Coinbase

Private Key vs Public Key

Point	Private key	Public key
Shared?	Never	Yes
Use	Sign transactions	Verify signatures; address derived from it
If leaked	Funds can be stolen	No risk by itself

Smart Contract vs Traditional Contract

Point	Smart contract	Traditional contract
Form	Code on blockchain	Written legal document
Enforcement	Automatic by code	By courts / parties
Intermediary	Not needed	Lawyers, courts
Change	Hard after deployment	By agreement

Ethereum vs Hyperledger

Point	Ethereum	Hyperledger Fabric
Type	Public, permissionless	Permissioned (enterprise)
Smart contracts	Solidity	Chaincode (Go, JS, Java)
Cryptocurrency	Ether is native	None required
Privacy	Public by default	Channels give private data sharing

Blockchain vs Traditional Database

Point	Blockchain	Traditional database
Control	Decentralised	Central administrator
Edit / delete	Append-only, hard to change	Create, read, update, delete
Speed	Slower	Fast
Trust	No single trusted party needed	Trust in the admin

Coin vs Token

Point	Coin	Token
Blockchain	Has its own (Bitcoin, Ether)	Built on another chain (ERC-20 tokens)
Created by	Protocol itself	Smart contract

Proof of Work vs Proof of Stake

Point	Proof of Work	Proof of Stake
Validators chosen by	Computing power (mining)	Amount of coin staked
Energy use	Very high	Much lower
Used by	Bitcoin	Ethereum (since 2022)

Gas Fee vs Transaction Value

Point	Gas fee	Transaction value
Meaning	Payment for computation to the network	Amount of Ether being sent
Paid to	Validators / burned (base fee)	Receiver
Example	21,000 gas × gas price	Sending 0.1 ETH

6. Important Formulas and Complexity Cheat Sheet

Group A: Algorithms

Topic	Formula / Fact	Meaning of symbols
Fibonacci	**F(n) = F(n-1) + F(n-2)**, F(0)=0, F(1)=1	Recursive: O(2^n) time, O(n) space. Iterative: O(n) time, O(1) space. Fast doubling: O(log n)
Huffman	Merge two smallest frequencies; **avg length = Σ f(i)·len(i) / Σ f(i)**	f(i) = frequency, len(i) = code length. Time O(n log n)
Fractional Knapsack	Maximise **Σ v(i)·x(i)** subject to **Σ w(i)·x(i) ≤ W**, 0 ≤ x(i) ≤ 1	v = value, w = weight, x = fraction taken. Sort by v/w, O(n log n)
0/1 Knapsack	Same objective with **x(i) ∈ {0,1}**. dp[i][w] = max(dp[i-1][w], v(i) + dp[i-1][w-w(i)])	Time and space O(nW). B&B worst case O(2^n)
N-Queens	Attack if **same column or |r1-r2| = |c1-c2|**	Backtracking, worst case O(N!). Solutions: N=4 → 2, N=8 → 92
Quick Sort	T(n) = T(k) + T(n-k-1) + O(n)	Best/average **O(n log n)**, worst **O(n²)**, space O(log n) average
Merge Sort	T(n) = 2T(n/2) + O(n)	**O(n log n)** all cases, space O(n), stable
Matrix multiplication	**C[i][j] = Σ A[i][k]·B[k][j]**	Standard O(n³); Strassen about O(n^2.81)
Naive matching	Compare at each of n−m+1 shifts	O(nm) worst, space O(1)
Rabin-Karp	**t(s+1) = (d·(t(s) − T[s]·h) + T[s+m]) mod q**, h = d^(m-1) mod q	d = radix, q = prime, m = pattern length. Average O(n+m), worst O(nm)
TSP	Brute force **(n−1)!**; Held-Karp **O(n²·2^n)**; MST approximation ratio **2**	Triangle inequality d(a,c) ≤ d(a,b)+d(b,c) needed for the ratio

Group B: Machine Learning

Topic	Formula / Fact	Meaning of symbols
Gradient Descent	**x_new = x_old − α·f′(x_old)**	α = learning rate, f′ = derivative (gradient)
Euclidean distance (KNN)	**d = √(Σ (x(i) − y(i))²)**	x, y = two data points with i features
Manhattan distance	**d = Σ |x(i) − y(i)|**	Sum of absolute differences
Linear Regression	**y = b0 + b1·x1 + … + bn·xn**	b0 = intercept, b = coefficients, x = features
Random Forest	Many trees on bootstrap samples; **average (regression) or majority vote (classification)**	Random feature subset at each split
Confusion matrix	TP, TN, FP, FN	TP = correct positive, TN = correct negative, FP = false alarm, FN = missed positive
Accuracy / Error	**(TP+TN)/(TP+TN+FP+FN)**; error = 1 − accuracy	Fraction predicted correctly
Precision / Recall / F1	**TP/(TP+FP)**; **TP/(TP+FN)**; **2PR/(P+R)**	P = precision, R = recall
R²	**R² = 1 − Σ(y−ŷ)² / Σ(y−ȳ)²**	y = actual, ŷ = predicted, ȳ = mean of actual
RMSE / MAE	**RMSE = √(Σ(y−ŷ)²/n)**; **MAE = Σ|y−ŷ|/n**	n = number of samples
K-Means	Minimise **WCSS = Σ over clusters Σ ‖x − μ‖²**	μ = cluster centroid. Elbow: plot WCSS vs k
Activation functions	Sigmoid **σ(z) = 1/(1+e^(−z))**; ReLU **max(0, z)**; softmax gives class probabilities	z = weighted sum of inputs
Loss functions	Binary cross-entropy **−[y·log(p) + (1−y)·log(1−p)]**; MSE for regression	y = true label (0/1), p = predicted probability
Min-Max / Z-score	**(x − min)/(max − min)**; **(x − μ)/σ**	μ = mean, σ = standard deviation

Group C: Blockchain

Topic	Formula / Fact	Meaning
Block hash	**Hash = SHA-256(previous hash + data + timestamp + nonce)**	Changing any input changes the hash. Ethereum uses Keccak-256
Block structure	Header (prev hash, Merkle root, timestamp, nonce) + transactions	Linked by previous-hash field
Transaction fee (Ethereum)	**Fee = Gas used × Gas price**, where gas price = base fee + priority fee	Gas price in gwei. Plain ETH transfer = 21,000 gas
Units	**1 ETH = 10^18 wei; 1 gwei = 10^9 wei**	Smallest unit is wei
Gas limit	Maximum gas you allow for one transaction	Unused gas is refunded; out-of-gas fails the transaction
Fee depends on	Network, congestion and transaction complexity	Test-network ETH has no real value; mainnet fees are real

7. Practical Demonstration Questions

What to Do If the Examiner Asks You to Explain Your Program

Speak in this order: Aim, Input, Algorithm or model, Main steps, Output, Complexity or metric, Conclusion. Use the template for your practical and replace details with your own.

A1. Fibonacci
Aim: To compare recursive and non-recursive Fibonacci.
Input: a number n.
Algorithm / model: recursion and a loop.
Main steps: check base cases, compute F(n) both ways, time both.
Expected output: first n terms, e.g. 0 1 1 2 3 5 8.
Complexity / metric: recursive O(2^n) time and O(n) space; iterative O(n) time and O(1) space.
Conclusion: the iterative method is far faster; the recursive one repeats subproblems.

A2. Huffman Encoding
Aim: To compress text using Huffman coding.
Input: a string or character frequencies.
Algorithm / model: greedy, using a min-heap.
Main steps: count frequencies, build the tree by merging two smallest nodes, assign 0/1 codes, encode the text.
Expected output: code for each character and the encoded bit string.
Complexity / metric: O(n log n) time, O(n) space.
Conclusion: frequent characters get shorter codes, so total bits decrease.

A3. Fractional Knapsack
Aim: To maximise value in a bag when items can be split.
Input: values, weights and capacity W.
Algorithm / model: greedy by value/weight ratio.
Main steps: compute ratios, sort descending, take whole items, then a fraction of the last one.
Expected output: maximum value and fraction of each item taken.
Complexity / metric: O(n log n).
Conclusion: greedy is optimal because items are divisible.

A4. 0/1 Knapsack
Aim: To find maximum value when each item is taken whole or not at all.
Input: values, weights, capacity W.
Algorithm / model: dynamic programming (or branch and bound).
Main steps: build dp table using dp[i][w] = max(skip, take), read dp[n][W], backtrack for items.
Expected output: maximum value and chosen items.
Complexity / metric: O(nW) time and space for DP.
Conclusion: DP finds the optimum where greedy fails.

A5. N-Queens
Aim: To place N queens so none attack each other.
Input: board size N.
Algorithm / model: backtracking.
Main steps: place a queen row by row, check column and diagonal safety, backtrack when stuck.
Expected output: board with Q marks, or a message if no solution.
Complexity / metric: worst case O(N!), space O(N).
Conclusion: backtracking prunes invalid branches early.

A6. Quick Sort
Aim: To implement deterministic and randomized Quick Sort.
Input: an array of numbers.
Algorithm / model: divide and conquer using partition.
Main steps: choose pivot (fixed or random), partition, recurse on both sides.
Expected output: sorted array and running times.
Complexity / metric: best/average O(n log n), worst $O(n^2)$.
Conclusion: random pivot makes the worst case very unlikely on any input.

A7(a). Matrix Multiplication
Aim: To multiply matrices sequentially and with threads.
Input: two compatible matrices.
Algorithm / model: triple loop; one thread per row or block.
Main steps: check dimensions, compute rows in threads, join, compare time.
Expected output: product matrix and both times.
Complexity / metric: $O(n^3)$; ideal parallel $O(n^3/p)$.
Conclusion: threads help for large matrices; overhead hurts small ones.

A7(b). Merge Sort
Aim: To implement Merge Sort and multithreaded Merge Sort.
Input: an array.
Algorithm / model: divide and conquer; threads for halves.
Main steps: split, sort halves (in threads), merge.
Expected output: sorted array and times.
Complexity / metric: O(n log n), space O(n).
Conclusion: multithreading speeds up sorting but the merge stays sequential.

A7(c). String Matching
Aim: To find a pattern in a text using naive and Rabin-Karp.
Input: text and pattern.
Algorithm / model: sliding window; rolling hash.
Main steps: naive: compare each shift; Rabin-Karp: compare hashes then verify.
Expected output: starting indices of matches.
Complexity / metric: naive O(nm); Rabin-Karp average O(n+m).
Conclusion: Rabin-Karp is faster on average but needs collision checking.

A7(d). TSP
Aim: To find the shortest tour through all cities.
Input: distance matrix.
Algorithm / model: exact (brute force or DP) and MST-based approximation.
Main steps: compute exact tour; build MST, preorder walk, shortcut; compare.
Expected output: tour order and total cost for both.
Complexity / metric: exact exponential; approximation at most 2× optimal for metric TSP.
Conclusion: approximation is faster with a bounded loss in quality.

B1. Uber Fare Prediction
Aim: To predict fare_amount from trip details.
Input: Uber trips dataset.
Algorithm / model: Linear Regression and Random Forest Regression.
Main steps: clean data, remove outliers, compute distance, check correlation, split, train, evaluate.
Expected output: predicted fares and metric values.
Complexity / metric: R^2 and RMSE.
Conclusion: the model with higher test R^2 and lower RMSE is preferred.

B2. Email Spam Classification
Aim: To classify emails as spam or not spam.
Input: email dataset with word features.
Algorithm / model: KNN and SVM.
Main steps: preprocess, scale, split, train both models, compare using confusion matrix.
Expected output: predicted classes and metrics.
Complexity / metric: accuracy, precision, recall, F1.
Conclusion: choose the model with better precision and recall on test data.

B3. Bank Churn Prediction
Aim: To predict whether a customer will leave.
Input: bank customer dataset.
Algorithm / model: neural network (Dense layers, ReLU, sigmoid).
Main steps: drop IDs, encode, scale, split, build and train network, evaluate.
Expected output: churn probability and class.
Complexity / metric: accuracy, confusion matrix, recall, F1.
Conclusion: the network predicts churn; recall matters for retention.

B4. Gradient Descent
Aim: To find the local minimum of a function.
Input: function, starting point, learning rate.
Algorithm / model: gradient descent $x = x - \alpha \cdot f'(x)$.
Main steps: compute gradient, update x, repeat until change is tiny.
Expected output: value of x at minimum and f(x).
Complexity / metric: number of iterations and final gradient near 0.
Conclusion: the learning rate controls speed and stability.

B5. KNN on Diabetes Data
Aim: To classify patients as diabetic or not.
Input: diabetes dataset (8 features).
Algorithm / model: KNN.
Main steps: replace invalid zeros, scale, split, choose k, predict, evaluate.
Expected output: predicted outcomes.
Complexity / metric: confusion matrix, accuracy, error rate, precision, recall.
Conclusion: recall is vital in medical screening.

B6. K-Means and Hierarchical Clustering
Aim: To group data into clusters.
Input: numeric dataset (no labels).
Algorithm / model: K-Means with the elbow method; agglomerative clustering with a dendrogram.
Main steps: scale, run elbow to choose k, cluster, plot dendrogram.
Expected output: cluster labels and plots.
Complexity / metric: WCSS and silhouette score.
Conclusion: clustering reveals natural groups such as customer segments.

B7. Stock Market Analysis
Aim: To analyse and predict stock prices.
Input: historical OHLCV data.
Algorithm / model: moving averages, regression or LSTM.
Main steps: load data, plot trends, scale, create windows, split by time, train, evaluate.
Expected output: actual versus predicted price plot.
Complexity / metric: RMSE and MAE.
Conclusion: the model learns past patterns but cannot guarantee future prices.

B8. Titanic Survival
Aim: To predict passenger survival.
Input: Titanic dataset.
Algorithm / model: Logistic Regression / Random Forest.
Main steps: fill missing Age and Embarked, drop Cabin, encode, split, train, evaluate.
Expected output: survived or not.
Complexity / metric: accuracy and confusion matrix.
Conclusion: sex, class and age are influential features (verify using EDA).

B9. Signature Identification
Aim: To identify the signer from a signature image.
Input: custom signature image dataset.
Algorithm / model: CNN (or SVM on image features).
Main steps: collect images, grayscale, resize, normalise, augment, train, test.
Expected output: predicted person (or genuine/forged).
Complexity / metric: accuracy, confusion matrix, FAR and FRR.
Conclusion: a custom dataset needs enough samples and augmentation.

C1. MetaMask and Ether per Transaction
Aim: To install MetaMask and study the cost of a transaction.
Input: test Ether from a faucet.
Algorithm / model: Ethereum test network transactions.
Main steps: install, create wallet, switch to test network, send a transaction, view it on Etherscan.
Expected output: transaction hash, gas used and fee.
Complexity / metric: fee = gas used × gas price.
Conclusion: fees depend on gas used and network congestion.

C2. Create Wallet in MetaMask
Aim: To create a cryptocurrency wallet.
Input: password and secret recovery phrase.
Algorithm / model: key-pair based wallet.
Main steps: install, create wallet, back up the seed phrase, view address.
Expected output: wallet address starting with 0x.
Complexity / metric: not applicable (security: keep seed phrase secret).
Conclusion: the wallet holds keys, not coins; the seed phrase must be protected.

C3. Bank Smart Contract
Aim: To create a bank account contract in Solidity.
Input: Ether amounts for deposit and withdraw.
Algorithm / model: Solidity contract with payable deposit, withdraw and balance.
Main steps: write code in Remix, compile, deploy, call functions.
Expected output: updated balance after each call.
Complexity / metric: gas used per function.
Conclusion: smart contracts enforce rules automatically on the blockchain.

C4. Student Data Contract
Aim: To store student records using struct and array.
Input: student id, name, marks.
Algorithm / model: Solidity struct, array and fallback function.
Main steps: define struct, push to array, read by index, test fallback, deploy, check gas.
Expected output: student details and gas data.
Complexity / metric: fee = gas used × gas price.
Conclusion: storing data on-chain costs gas; reads are free.

C5. Types of Blockchain
Aim: To study blockchain types and uses.
Input: case studies.
Algorithm / model: comparison of public, private and consortium types.
Main steps: list types, compare, map to use cases.
Expected output: comparison table and examples.
Complexity / metric: not applicable.
Conclusion: choose the type by need for openness, privacy and speed.

C6. Hyperledger Business Network
Aim: To create a business network.
Input: organisations, assets and participants.
Algorithm / model: Hyperledger Fabric (or Composer).
Main steps: define organisations and channel, deploy chaincode, submit transactions.
Expected output: transactions on the shared ledger.
Complexity / metric: not applicable.
Conclusion: permissioned networks suit enterprises needing privacy.

C7. E-Voting dApp
Aim: To build a transparent voting application.
Input: candidates and voters.
Algorithm / model: Solidity contract with web front end.
Main steps: deploy contract, register voters, vote through MetaMask, display results.
Expected output: vote counts.
Complexity / metric: gas per vote.
Conclusion: blockchain gives transparency; identity and privacy remain challenges.

C8. Charity Application
Aim: To make donations transparent.
Input: donor Ether payments.
Algorithm / model: Solidity contract with donate and release functions.
Main steps: deploy, donate, record events, approve spending.
Expected output: donation records and balances.
Complexity / metric: gas per transaction.
Conclusion: public records increase donor trust.

C9. Medical Records App
Aim: To manage medical records securely.
Input: patient and doctor addresses, record hashes.
Algorithm / model: contract with access control; off-chain encrypted storage.
Main steps: add record hash, grant and revoke access, log access.
Expected output: permission checks and record pointers.
Complexity / metric: gas per transaction.
Conclusion: store hashes on-chain and data off-chain for privacy.

C10. Mental Health App
Aim: To provide private, consent-based records and support.
Input: user, therapist and consent data.
Algorithm / model: contract for consent and verification; encrypted off-chain storage.
Main steps: register users, verify therapists, grant consent, book sessions.
Expected output: consent and appointment records.
Complexity / metric: gas per transaction.
Conclusion: privacy and consent are the key design points; it does not replace professional help.

Common Follow-up Questions

D-1 Why did you use this algorithm or model?

Because it suits the problem type: for example greedy for fractional knapsack, DP for 0/1 knapsack, backtracking for N-Queens, Random Forest for non-linear regression. Mention one advantage and the complexity or metric.

D-2 Why did you choose this dataset?

It is the dataset specified in the practical or a standard public one, it has meaningful features for the target, and it is of manageable size. Mention rows, features and target.

D-3 How did you test your program?

With small inputs I could verify by hand, edge cases (empty, one element, duplicates), and then larger inputs. For ML, I used a held-out test set and metrics.

D-4 What happens if the input is invalid?

The program should validate input and show a clear error message, for example negative n for Fibonacci, mismatched matrix sizes, or missing values in a dataset.

D-5 How can you improve the implementation?

Algorithms: memoization, pruning, better pivot or hash choice, thread pools. ML: more data, feature engineering, tuning with cross-validation. Blockchain: gas optimisation, security checks.

D-6 What is the most important observation from the output?

State one clear fact: for example the iterative Fibonacci is much faster; the Random Forest has higher R^2 than Linear Regression; deployment costs more gas than a read call.

D-7 What are the limitations of your work?

Small datasets, limited test cases, results depending on hardware or network, and simplified assumptions. Be honest, then say how you would extend it.

D-8 Can you explain the code line by line?

Prepare to explain the main function, the loop or recursion, and the final output. Know what every variable and library call does.

Questions for Mini Projects

M-1 What is the architecture of your mini project?

Input module → algorithm (sequential and parallel/approximate versions) → timer → output and comparison. For blockchain: front end → wallet → smart contract → blockchain network.

M-2 Describe the workflow of your project.

Take input, validate, run the algorithm, record results and time, and display output. Mention the order of steps in simple words.

M-3 Why did you make these implementation choices?

Say the reason: language availability, simplicity, library support, and the need to compare approaches fairly on the same input.

M-4 How did you test the mini project?

Used the same inputs for all versions, compared outputs for equality, tried different input sizes, and repeated timing several times to avoid noise.

M-5 How did you compare performance?

Measured running time for increasing input sizes, tabulated results and computed speed-up = time(sequential) / time(improved).

M-6 What are the limitations of your mini project?

Results depend on hardware and language, thread overhead for small inputs, approximation does not give optimum, and the blockchain app runs only on a test network.

8. Top 50 Most Important Viva Questions

Selected as highly likely topics across all groups. Tags: DAA, ML, Blockchain. These are short revision answers; full explanations are in the group sections.

T-1 [DAA] What is the time complexity of recursive Fibonacci and why?
O(2^n), because each call makes two more calls and subproblems repeat.

T-2 [DAA] How can Fibonacci be computed in O(n) time and O(1) space?
Use a loop keeping only the last two values.

T-3 [DAA] What is the greedy strategy?
Pick the best local choice at each step, hoping to reach a global optimum.

T-4 [DAA] How does Huffman coding work?
Repeatedly merge the two least frequent nodes using a min-heap; assign 0 to left and 1 to right edges.

T-5 [DAA] What is the complexity of Huffman coding?
O(n log n).

T-6 [DAA] Which sorting criterion does Fractional Knapsack use?
Value-to-weight ratio, highest first.

T-7 [DAA] Why does greedy fail for 0/1 Knapsack?
Items cannot be split, so a good-ratio item can waste capacity.

T-8 [DAA] Write the 0/1 Knapsack DP recurrence.
dp[i][w] = max(dp[i-1][w], v(i) + dp[i-1][w - w(i)]).

T-9 [DAA] What is the complexity of DP knapsack?
O(nW) time and space (pseudo-polynomial).

T-10 [DAA] What is backtracking?
Build a solution step by step and undo a step when it leads to a dead end.

T-11 [DAA] How do you check diagonal conflict in N-Queens?
Queens attack if $|r1 - r2| = |c1 - c2|$.

T-12 [DAA] How many solutions does 8-Queens have?
92.

T-13 [DAA] What is the worst case of Quick Sort and when does it occur?
$O(n^2)$, with already sorted input and a first or last pivot.

T-14 [DAA] Why use a randomized pivot?
It makes the expected time O(n log n) for every input.

T-15 [DAA] Is Merge Sort stable? What is its space?
Yes, stable. Space O(n).

T-16 [DAA] Complexity of standard matrix multiplication?
$O(n^3)$.

T-17 [DAA] Why can multithreading give no speed-up?
Thread overhead, few cores, small input or sequential parts like the merge step.

T-18 [DAA] What is a rolling hash in Rabin-Karp?
A hash updated in O(1) when the window slides by one character.

T-19 [DAA] What is a spurious hit?
Equal hash values for different strings; must be checked by comparing characters.

T-20 [DAA] Why is TSP hard and what is the approximation ratio of the MST method?
TSP is NP-hard. The MST method gives ratio 2 for metric TSP.

T-21 [ML] What are overfitting and underfitting?
Overfitting: good on training, poor on test. Underfitting: poor on both.

T-22 [ML] Why split into train and test sets?
To measure performance on unseen data.

T-23 [ML] How do you detect outliers?
Boxplot / IQR rule or z-score.

T-24 [ML] Which metrics are used for regression?
R^2, RMSE and MAE.

T-25 [ML] Write the formulas for precision and recall.
Precision = TP / (TP + FP); Recall = TP / (TP + FN).

T-26 [ML] What is a confusion matrix?
A table of TP, TN, FP and FN counts for a classifier.

T-27 [ML] How does Random Forest work?
Many decision trees on random samples; predictions are averaged or voted.

T-28 [ML] How does KNN classify?
By the majority class among the k nearest training points.

T-29 [ML] How do you choose k in KNN?
Try several values and pick the one with the lowest validation error; prefer odd k.

T-30 [ML] What is the SVM idea?
Find the maximum-margin hyperplane; support vectors lie on the margin.

T-31 [ML] Why is scaling needed?
Distance-based and gradient-based models are affected by feature ranges.

T-32 [ML] What is the output activation for churn prediction?
Sigmoid with 1 neuron, with binary cross-entropy loss.

T-33 [ML] Write the Gradient Descent update rule.
$x = x - \alpha \cdot f'(x)$.

T-34 [ML] What happens if the learning rate is too large?
The steps overshoot and the algorithm may diverge.

T-35 [ML] What is the elbow method?
Plot WCSS against k and choose k at the bend of the curve.

T-36 [ML] K-Means vs Hierarchical in one line?
K-Means needs k in advance; hierarchical gives a dendrogram and k is chosen later.

T-37 [ML] Why split stock data by time?
To avoid data leakage from the future.

T-38 [ML] Which Titanic columns have missing values?
Age, Cabin and Embarked (in the standard training file).

T-39 [ML] Identification vs verification in signatures?
Identification asks who signed; verification asks whether it is genuine.

T-40 [Blockchain] What is a blockchain?
A distributed ledger of hash-linked blocks shared by many nodes.

T-41 [Blockchain] What does a block contain?
Transactions, timestamp, previous hash, own hash and nonce.

T-42 [Blockchain] What is consensus?
The method by which nodes agree on the valid ledger, such as PoW or PoS.

T-43 [Blockchain] What is MetaMask?
A wallet and browser extension to interact with Ethereum applications.

T-44 [Blockchain] What is a seed phrase?
12 or 24 words that can restore all your keys; never share it.

T-45 [Blockchain] Formula for transaction fee?

Gas used × gas price.

T-46 [Blockchain] What is a smart contract?

A program on the blockchain that runs automatically by its rules.

T-47 [Blockchain] What is a fallback function?

A function that runs when an unknown function is called or Ether is sent with no matching function.

T-48 [Blockchain] Public vs private vs consortium blockchain?

Open to all, controlled by one organisation, controlled by a group of organisations.

T-49 [Blockchain] What is Hyperledger Fabric?

A permissioned enterprise blockchain framework from the Linux Foundation.

T-50 [Blockchain] Why not store large medical data on-chain?

It is costly, public and permanent; store a hash on-chain and data off-chain.

9. Rapid-Fire One-Line Questions and Answers

Group A: DAA

#	Question	Answer
R-1	Full form of DAA?	Design and Analysis of Algorithms.
R-2	Fibonacci recurrence?	F(n) = F(n-1) + F(n-2).
R-3	Recursive Fibonacci time?	O(2^n).
R-4	Iterative Fibonacci space?	O(1).
R-5	Which structure does Huffman use?	Min-heap (priority queue).
R-6	Is Huffman lossy?	No, lossless.
R-7	Number of internal nodes in Huffman tree with n leaves?	n − 1.
R-8	Fractional Knapsack strategy?	Greedy.
R-9	0/1 Knapsack strategy?	Dynamic programming.
R-10	Fractional Knapsack time?	O(n log n).
R-11	DP knapsack time?	O(nW).
R-12	Brute-force knapsack time?	O(2^n).
R-13	N-Queens strategy?	Backtracking.
R-14	Queens in same diagonal if?	|r1 − r2| = |c1 − c2|.
R-15	Solutions of 4-Queens?	2.
R-16	Quick Sort worst case?	$O(n^2)$.
R-17	Quick Sort average case?	O(n log n).
R-18	Is Quick Sort stable?	No.
R-19	Is Quick Sort in-place?	Yes.
R-20	Is Merge Sort stable?	Yes.
R-21	Merge Sort time (all cases)?	O(n log n).
R-22	Merge Sort extra space?	O(n).
R-23	Matrix multiplication time?	$O(n^3)$.
R-24	Strassen's complexity?	About O(n^2.81).
R-25	Rows computed by each thread?	One row or a block of rows.
R-26	Race condition?	Two threads changing shared data without control.
R-27	Naive matching worst case?	O(nm).
R-28	Rabin-Karp average case?	O(n + m).
R-29	Rabin-Karp worst case?	O(nm).
R-30	Another linear matching algorithm?	KMP.
R-31	TSP stands for?	Travelling Salesperson Problem.
R-32	Is TSP NP-hard?	Yes.
R-33	Brute-force TSP time?	O((n−1)!).
R-34	Held-Karp time?	$O(n^2 \cdot 2^n)$.
R-35	MST approximation ratio for TSP?	2 (metric TSP).
R-36	Christofides ratio?	1.5.
R-37	Divide and conquer examples?	Merge Sort, Quick Sort.
R-38	Backtracking is based on which search?	Depth-first search.
R-39	What does 'pruning' mean?	Cutting off branches that cannot lead to a solution.

#	Question	Answer
R-40	Optimal substructure means?	The optimal solution contains optimal solutions of subproblems.
R-41	Overlapping subproblems means?	The same subproblems are solved repeatedly.

Group B: Machine Learning

#	Question	Answer
R-42	Supervised learning needs?	Labelled data.
R-43	Clustering is which type?	Unsupervised.
R-44	Target of Uber dataset?	fare_amount.
R-45	Formula to get distance from coordinates?	Haversine formula.
R-46	Range of correlation?	−1 to +1.
R-47	Best possible R^2?	1.
R-48	RMSE unit?	Same as the target variable.
R-49	Random Forest is based on?	Decision trees.
R-50	Does Random Forest need scaling?	No.
R-51	KNN is a lazy learner. Why?	It does no training and computes at prediction time.
R-52	SVM data points on margin?	Support vectors.
R-53	Spam: costlier error?	False positive (real mail marked as spam).
R-54	Output neuron for binary churn?	1 neuron with sigmoid.
R-55	ReLU formula?	max(0, z).
R-56	Optimizer commonly used?	Adam.
R-57	Loss for binary classification?	Binary cross-entropy.
R-58	Gradient descent moves?	Opposite to the gradient.
R-59	Learning rate symbol?	α (alpha).
R-60	Number of rows in Pima diabetes dataset?	768.
R-61	Diabetes: important metric?	Recall.
R-62	Invalid zeros in diabetes data are in?	Glucose, BloodPressure, SkinThickness, Insulin, BMI.
R-63	K-Means objective?	Minimise WCSS.
R-64	Elbow method plots?	WCSS versus k.
R-65	Dendrogram belongs to?	Hierarchical clustering.
R-66	K-Means initial centroid improvement?	k-means++.
R-67	Stock data type?	Time series.
R-68	LSTM stands for?	Long Short-Term Memory.
R-69	Titanic target column?	Survived.
R-70	Titanic column dropped for many missing values?	Cabin.
R-71	FamilySize formula?	SibSp + Parch + 1.
R-72	Signature model commonly used?	CNN.
R-73	FAR stands for?	False Acceptance Rate.
R-74	FRR stands for?	False Rejection Rate.
R-75	Accuracy formula?	(TP + TN) / total.
R-76	F1-score formula?	2PR / (P + R).
R-77	One-hot encoding creates?	A 0/1 column for each category.
R-78	Normalization range?	0 to 1.
R-79	Standardization gives?	Mean 0 and standard deviation 1.
R-80	Dropout is used to?	Reduce overfitting.
R-81	Epoch means?	One full pass over the training data.

Group C: Blockchain

#	Question	Answer
R-82	Hash used by Bitcoin?	SHA-256.
R-83	First block called?	Genesis block.
R-84	Nonce is used in?	Proof of Work mining.
R-85	Unit smaller than gwei?	Wei.
R-86	1 ETH in wei?	10^18.
R-87	1 gwei in wei?	10^9.
R-88	Gas for simple ETH transfer?	21,000.
R-89	Fee formula?	Gas used × gas price.
R-90	Language of Ethereum smart contracts?	Solidity.
R-91	Online IDE for Solidity?	Remix IDE.
R-92	Test network commonly used?	Sepolia.
R-93	Does test Ether have real value?	No.
R-94	Wallet address starts with?	0x.
R-95	Who should know the private key?	Only the owner.
R-96	Seed phrase length?	12 or 24 words.
R-97	Function to receive Ether?	payable function.
R-98	Keyword to check a condition?	require.
R-99	Who called the function?	msg.sender.
R-100	Ether sent with call?	msg.value.
R-101	Read-only function keyword?	view.
R-102	Permanent data location?	storage.
R-103	Temporary data location?	memory.
R-104	Fallback signature?	fallback() external payable.
R-105	Plain Ether receive function?	receive() external payable.
R-106	Attack on withdraw functions?	Reentrancy.
R-107	Out of gas means?	Gas limit was too low for the work.
R-108	Ethereum consensus today?	Proof of Stake.
R-109	Bitcoin consensus?	Proof of Work.
R-110	Hyperledger host?	Linux Foundation.
R-111	Hyperledger smart contract name?	Chaincode.
R-112	Component that orders transactions in Fabric?	Orderer.
R-113	Is Fabric permissioned?	Yes.
R-114	dApp front-end libraries?	Web3.js or Ethers.js.
R-115	Where to store large files for dApps?	Off-chain, such as IPFS.
R-116	Etherscan is?	A block explorer.
R-117	Tool that views transaction hash details?	Block explorer.
R-118	Blockchain type for banks consortium?	Consortium.
R-119	Bitcoin is which type?	Public.
R-120	Digital signing uses?	Private key.
R-121	Merkle root stored in?	Block header.

10. Last-Minute Revision Sheet

Algorithm complexity at a glance

- Fibonacci: recursive **O(2^n)**, iterative **O(n)**, space O(n) vs O(1).
- Huffman **O(n log n)**; Fractional Knapsack **O(n log n)**; 0/1 Knapsack DP **O(nW)**.
- N-Queens backtracking: worst **O(N!)**.
- Quick Sort: best/avg **O(n log n)**, worst **O(n²)**; Merge Sort **O(n log n)**, space O(n).
- Matrix multiplication **O(n³)**; Strassen about O(n^2.81).
- Naive matching **O(nm)**; Rabin-Karp avg **O(n+m)**, worst O(nm).
- TSP: brute force **(n−1)!**, Held-Karp **O(n²·2^n)**, MST approximation ratio **2**.

Machine learning at a glance

- Regression metrics: **R², RMSE, MAE**. Classification: **accuracy, precision, recall, F1, confusion matrix**.
- KNN and SVM need **scaling**; Random Forest does not.
- Gradient Descent: **x = x − α·f′(x)**.
- K-Means: minimise **WCSS**; choose k using the **elbow method**.
- Churn: ANN with ReLU hidden layers, **sigmoid** output, **binary cross-entropy**, Adam.
- Stock data: split **by time**; Titanic: fill Age and Embarked, drop Cabin; Diabetes: fix invalid zeros.
- Signature: grayscale, resize, normalise, augment, CNN; metrics include FAR and FRR.

Blockchain at a glance

- Block = data + timestamp + previous hash + hash + nonce.
- **Fee = gas used × gas price**; 1 ETH = 10^18 wei; 1 gwei = 10^9 wei; simple transfer = 21,000 gas.
- Never share the **private key or seed phrase**. Test networks (such as Sepolia) use free test Ether; mainnet uses real money.
- Solidity keywords: **payable, require, msg.sender, msg.value, view, struct, mapping, fallback, receive**.
- Public vs private vs consortium; Ethereum (public, Solidity) vs Hyperledger Fabric (permissioned, chaincode).
- Store large or sensitive data **off-chain** and keep hashes and permissions on-chain.

Viva-day checklist

- Programs run; sample inputs and outputs ready; dataset file and notebook open.
- Know the aim, input, method, output, complexity or metric and conclusion for every practical.
- MetaMask unlocked on a test network with test Ether; contract address and transaction hashes noted.
- Be confident, speak in short clear sentences, and admit when you are unsure.

Good luck for your viva! Remember: understand, do not just memorise, and be honest about your own results.